# Practical Zero-Knowledge PIOP for Maliciously Secure Multiparty Homomorphic Encryption

Intak Hwang[1], Hyeonbum Lee[2], Jinyeong Seo[1], and Yongsoo Song[1]

[1] Seoul National University
{intak.hwang, jinyeong.seo, y.song}@snu.ac.kr
[2] Hanyang University
leehb3706@hanyang.ac.kr

**Abstract.** Homomorphic encryption (HE) is a foundational technology in privacy-enhancing cryptography, enabling computation over encrypted data. Recently, generalized HE primitives designed for multi-party applications, such as multi-party HE (MPHE), have garnered significant research interest. While constructing secure multi-party protocols from MPHE in the semi-honest model is straightforward, achieving malicious security remains challenging as it requires zero-knowledge arguments of knowledge (ZKAoKs) for MPHE ciphertexts and public keys.
In this work, we design practical ZKAoKs for MPHE that validate the well-formedness of public keys and ciphertexts. Specifically, we develop our ZKAoKs within the polynomial interactive oracle proof (PIOP) framework. To achieve this, we introduce novel optimization techniques that seamlessly integrate constraints for MPHE into the PIOP framework, enabling the design of PIOPs for validating all types of MPHE public keys, including relinearization and automorphism keys. To the best of our knowledge, our construction is the first ZKAoK for MPHE that validates automorphism keys.
We instantiate our PIOP using a lattice-based polynomial commitment scheme (PCS). When compared with the previous state-of-the-art construction, PELTA (ACM CCS 2023), our implementation achieves a 5.4x reduction in proof size, a 111x speed-up in proof generation, and a 768x improvement in verification time for validating the encryption key. In addition to the encryption key, we provide benchmark results for all types of ZKAoKs required for MPHE, presenting the first concrete performance results in compiling passively secure MPHE-based protocols into maliciously secure ones.

**Keywords:** homomorphic encryption; zero-knowledge argument of knowledge; multiparty computation; malicious security

## 1 Introduction

Homomorphic encryption (HE) is an encryption scheme that enables computation on ciphertexts without decryption. One direct application of HE is secure two-party computation in a server-client setting. In particular, it provides optimal round complexity and low communication overhead, depending

only on the size of the client's input. Due to this property, homomorphic encryption has been widely adopted in various privacy-preserving protocols over the last decade. For example, it has been deployed in private information retrieval [ACLS18, MW22], private set intersection [CHLR18, CMdG+21], oblivious message retrieval [LT22, LTW24], and secure inference [LLL+22, JPK+24], demonstrating its versatility and practicality.

Multi-party homomorphic encryption (MPHE) refers to a variant of homomorphic encryption that extends its functionality to support multiple parties. Specifically, conventional HE allows homomorphic computation only between ciphertexts encrypted under the same key. As a result, it does not support computation on ciphertexts encrypted under different keys, which limits the scalability of protocols as the number of participants increases. To overcome this limitation, several approaches for constructing MPHE have been studied [AJLA+12, LATV12, MTPBH21, KKL+23, KLSW24]. One widely adopted method is to generate a shared public key among the participants, so that input ciphertexts are encrypted under the same key. This enables homomorphic computation between them. Based on MPHE, it is possible to construct general multi-party computation (MPC) protocols [AJLA+12, MCPT24], whose communication cost depends only on the sizes of the inputs and outputs, inheriting the efficiency advantages of HE.

However, MPHE provides only semi-honest security when used to construct MPC protocols, exposing vulnerabilities to malicious adversaries. In particular, malicious threats in MPHE-based protocols often arise from the decryption of malformed ciphertexts, which can compromise the simulatability of the protocol. This vulnerability has recently received significant attention in the context of the IND-CPA-D security model [LM21]. For example, concrete attacks [GNSJ24, CCP+24, CSBB24] on currently deployed HE schemes have been proposed within the IND-CPA-D framework, highlighting security risks in HE-based protocols against malicious adversaries.

Achieving malicious security in MPHE-based protocols has been explored from various directions [AJLA+12, MW16, CO17], but the most viable approach is the general compilation method in [AJLA+12], which integrates zero-knowledge arguments of knowledge (ZKAoKs) to verify the correct execution of the protocol, thus preventing malicious behavior. The compilation method effectively transforms passively secure MPHE-based protocols into maliciously secure ones at the cost of proof generation, which involves proving the well-formedness of MPHE ciphertexts and public keys. As a result, this naturally leads to the need for efficient ZKAoKs tailored to these components, and thus a line of research [AJLA+12, CMS+23] has focused on constructing efficient ZKAoKs for MPHE ciphertexts and public keys. However, these solutions remain far from practical deployment due to significant overheads in proof generation time or proof size.

### 1.1 Our Contributions

In this work, we design efficient zero-knowledge arguments of knowledge (ZKAoKs) for MPHE ciphertexts and public keys, which are required for achieving malicious security in MPHE-based protocols following the compilation method in [AJLA+12].

**Overview on PELTA.** We begin with a brief review of PELTA [CMS+23], the state-of-the-art ZKAoKs for MPHE, to motivate and explain our improvements. The main goal of PELTA is to achieve practical performance for proof generation. To this end, it leverages the LANES framework [ALS20, ENS20, LNS20], which provides fast proof generation based on lattice-based cryptography, at the cost of a large proof size that scales linearly with the input size.

However, the LANES framework is optimized for a small prime modulus because it is originally designed for lattice-based signatures. As a result, when applied to MPHE, where the ciphertext modulus is the product of many small primes, the framework requires dozens of repetitions to generate proofs. This leads to impractical performance when working with large HE parameters. In addition, PELTA does not address all components of MPHE public keys, omitting ZKAoKs for automorphism keys, which are necessary to support homomorphic automorphisms. This omission is critical, as homomorphic automorphisms are essential for homomorphic matrix multiplication, a frequently used operation in MPHE-based protocols [CKR+20]. Finally, the experimental results in PELTA focus on the performance of encryption and decryption of MPHE ciphertexts, but do not report performance metrics for ZKAoKs on the full set of public key components. This leaves the practicality of the overall solution unresolved.

**Overview on Our ZKAoKs.** Compared to PELTA, we design ZKAoKs for MPHE based on the polynomial interactive oracle proof (PIOP) framework [CHM+20, BFS20]. We choose to utilize the PIOP framework instead of the LANES framework because it provides higher expressiveness and sublinear proof sizes, which are essential given that MPHE public keys often have hundreds of megabyte sizes and their validity involves complex algebraic relations. The change of the base proof system allows us to successfully prove all required relations for the validity of MPHE public keys and ciphertexts while maintaining compact proof size.

In addition, we improve the performance of proof generation by devising a new technique that temporarily switches the ciphertext modulus to a single prime. As a result, we avoid repeated proof generation for each prime factor of the ciphertext modulus, which is a major performance bottleneck in PELTA.

To demonstrate the practicality of our PIOPs, we present a proof-of-concept implementation and provide benchmark results for all types of ZKAoKs required for MPHE. Notably, our implementation is the first to offer a complete benchmark for all types of ZKAoKs required for MPHE, bridging the gap between theory and practice in maliciously secure MPHE-based MPC protocols. Below, we describe our contributions in detail.

**Adaption of Univariate PIOP.** Our first contribution is the design of ZKAoKs for MPHE within the PIOP framework. To achieve this, we make the novel ob-

servation that univariate PIOPs from [BSCR+19, CHM+20] can be adapted to prove constraints for MPHE ciphertexts and public keys. Specifically, these univariate PIOPs were originally proposed for proving constraints over $\mathbb{Z}_p^N$, which is a vector space over the prime field $\mathbb{Z}_p$, to construct efficient SNARKs for R1CS relations. In contrast, constraints in MPHE are typically defined over the polynomial ring $R_p = \mathbb{Z}_p[X]/(X^N + 1)$, where $N$ is a power of two. This structural difference makes the direct application of univariate PIOPs non-trivial.

To address this inconsistency, we leverage the isomorphism $\mathbb{Z}_p^N \cong R_p$ when $2N \mid p - 1$. Since this isomorphism preserves the underlying algebraic structure, constraints defined over $R_p$ can be translated into constraints over $\mathbb{Z}_p^N$. By exploiting this translation, we successfully adapt the univariate PIOPs from [BSCR+19, CHM+20] to prove constraints defined over polynomial rings. Additionally, our design allows us to construct the first ZKAoK for validity of automorphism keys, thanks to the high expressiveness of univariate PIOPs.

**Novel Proof Technique via Modulus Switching.** While the PIOP framework enables efficient proof generation, inconsistency issues regarding the ciphertext modulus still remain. Specifically, MPHE ciphertexts and public keys are typically generated over the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$, where $q = \prod_{i=0}^{\ell-1} q_i$ is the product of distinct primes $q_i$, while the PIOP framework requires a single, large prime modulus. As a result, directly applying univariate PIOPs leads to generating separate proofs for each $R_{q_i}$ by leveraging the isomorphism $R_q \cong \prod_{i=0}^{\ell-1} R_{q_i}$. However, this approach significantly degrades performance across all aspects: proof generation, verification, and proof size.

To address this issue, we introduce a novel proof technique that incorporates the modulus switching operation [BV11] for efficient proof generation. Modulus switching is an operation for HE ciphertexts that allows the conversion of the ciphertext modulus while preserving the well-formedness of ciphertexts. Initially proposed for designing homomorphic multiplication, we repurpose this operation for efficient proof generation by temporarily switching the ciphertext modulus from $q$ to a prime $p \approx q$. By doing so, we can prove the constraints for well-formedness in a single polynomial ring $R_p$, rather than proving them separately in each $R_{q_i}$, effectively reducing performance overhead. While the proofs are generated over $R_p$, the modulus switching operation ensures that the well-formedness of ciphertexts and public keys is still maintained in $R_q$. Thus, we successfully resolve the modulus inconsistency issue by leveraging the modulus switching operation.

**Implementation.** To demonstrate the practicality of our PIOPs for MPHE, we implement them using the HSS polynomial commitment scheme [HSS24]. For benchmarking, we measure the proof size and the runtimes of both the prover and the verifier when validating various types of public keys. For encryption keys, our PIOP achieves a 5.5× smaller proof size, 111.3× faster proof generation, and 768.5× faster verification compared to the previous state-of-the-art construction by Chatel et al. [CMS+23]. For the full set of public key components, including encryption, relinearization, and automorphism keys, we achieve proof generation in 78.5 seconds and a proof size of 86MB, which is smaller than the entire public

key size of 273MB. This reduction is attributed to the sublinear complexity of the proof size in our ZKAoK construction. Additionally, to the best of our knowledge, this is the first benchmark that validates the well-formedness of the complete set of MPHE public keys, making our ZKAoKs practically deployable.

### 1.2 Applications

**Maliciously Secure MPC.** The most direct application of our PIOPs is to achieve malicious security in MPHE-based MPC protocols. It has been shown in [AJLA+12] that passively secure MPHE-based MPC protocols can be transformed into maliciously secure ones by simply attaching ZKAoKs for MPHE ciphertexts and public keys. Since our PIOPs support all the necessary ZKAoKs for MPHE, integrating proofs generated by our PIOPs directly enhances the security of existing MPHE-based MPC protocols without requiring significant modifications to the protocols.

**SPDZ Offline Phase.** The SPDZ [DPSZ12] protocol is a secret-sharing-based MPC protocol that achieves adaptive security against malicious adversaries. In the offline phase of SPDZ, correlated randomness, known as authenticated triples, is generated using MPHE functionality. While the literature provides efficient proof systems for verifying MPHE ciphertexts, a dedicated proof system for joint public keys in MPHE is not precisely described. Recently, Rotaru et al. [RST+22] propose an MPC-based solution for securely constructing these public keys, which requires about 47 minutes in the two-party setting. We observe that our PIOP for MPHE public keys offers an alternative solution to this problem, as it can generates all the required proofs for joint public keys in 91 seconds.

### 1.3 Related Work

**ZKAoKs for (MP)HE.** Besides PELTA [CMS+23], several works in the literature have previously constructed ZKAoKs for (MP)HE. Del Pino et al. [DPLS19] utilize the inner product argument from Bulletproofs [BBB+18] to represent constraints for HE ciphertexts, but they do not provide concrete performance metrics. Boschini et al. [BCOS20] construct a proof system based on the R1CS representation and generate proofs via Aurora [BSCR+19], which is also based on PIOPs. While this approach achieves a short proof size for HE ciphertexts due to its polylogarithmic complexity, it suffers from slow proof generation due to a lack of PIOP-level optimization.

**Verifiable HE.** In addition to ensuring the validity of HE ciphertexts and public keys, another line of research [BCFK21, FNP20, GNSV23], known as verifiable homomorphic encryption, focuses on validating the correct execution of homomorphic operations. While the goal of verifiable HE is orthogonal to our work, our methodology is highly relevant to this line of research. Specifically, verifiable HE leverages SNARKs to prove computational correctness, offering sublinear verification complexity in circuit size, and state-of-the-art SNARKs [CHM+20]

are often built on the PIOP framework. Consequently, we believe our PIOPs for MPHE can be extended to create efficient SNARKs for HE operations within the same framework.

## 2 Background

### 2.1 Notation

For a positive integer $q$, we use the set $\mathbb{Z} \cap \left(-\frac{q}{2}, \frac{q}{2}\right]$ as a representative set for $\mathbb{Z}_q$, and denote the reduction of an integer $a$ modulo $q$ by $[a]_q$. Vectors over $\mathbb{Z}$ or $\mathbb{Z}_q$ are denoted by regular lowercase letters with arrows, such as $\vec{v}$, while matrices over $\mathbb{Z}$ or $\mathbb{Z}_q$ are represented by regular uppercase letters. We treat all vectors as column vectors and use the symbol $\|$ to denote the concatenation of two vectors. The Hadamard product between vectors is denoted by $\circ$.

Let $N$ be a power of two. We define $R = \mathbb{Z}[X]/(X^N+1)$ as the ring of integers of the $2N$-th cyclotomic field, and $R_q = \mathbb{Z}_q[X]/(X^N+1)$ as the quotient ring of $R$ modulo $q$. Polynomials are denoted by bold lowercase letters, e.g., $\boldsymbol{f}$. For a vector $\vec{v} = (v_0, \ldots, v_{n-1}) \in \mathbb{Z}^n$, the $L^p$-norm ($p \geq 1$) and the $L^\infty$-norm are defined as $\|\vec{v}\|_p := \sqrt[p]{\sum_{i=0}^{n-1} |v_i|^p}$ and $\|\vec{v}\|_\infty := \max_{0 \leq i < n} |v_i|$. For a polynomial $\boldsymbol{f}$, $\|\boldsymbol{f}\|_p$ is defined as the norm of the vector of its coefficients. For a distribution $\mathcal{D}$, we denote sampling $x$ from $\mathcal{D}$ by $x \leftarrow \mathcal{D}$. For a polynomial $\boldsymbol{f}$ of degree less than $n$, we write $\boldsymbol{f} \leftarrow \mathcal{D}^n$ when each coefficient of $\boldsymbol{f}$ is sampled independently from $\mathcal{D}$. For a finite set $S$, the uniform distribution over $S$ is denoted by $\mathcal{U}(S)$.

### 2.2 Interactive Argument of Knowledge

We define an interactive argument of knowledge and its zero-knowledge version as follows.

**Definition 1 (Interactive Argument of Knowledge).** *Let $\Pi = (\mathtt{Setup}, \mathtt{P}, \mathtt{V})$ be an interactive protocol between a prover $\mathtt{P}$ and a verifier $\mathtt{V}$. $\Pi$ is called an argument of knowledge for a relation $\mathsf{R}$ if it satisfies the following properties.*

- ***Completeness**: For all PPT adversary $\mathtt{A}$, the following holds.*

$$\Pr\left[\left\langle \mathtt{P}(\mathsf{pp}, \mathsf{x}, \mathsf{w}), \mathtt{V}(\mathsf{pp}, \mathsf{x}) \right\rangle = 1 \vee (\mathsf{x}, \mathsf{w}) \notin \mathsf{R} \;\middle|\; \begin{array}{r} \mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda) \\ (\mathsf{x}, \mathsf{w}) \leftarrow \mathtt{A}(\mathsf{pp}) \end{array}\right] \geq 1 - \mathsf{negl}(\lambda)$$

- ***Knowledge Soundness**: For every PPT adversary $\mathtt{A} = (\mathtt{A}_1, \mathtt{A}_2)$, there exists a PPT extractor $\mathtt{E}$ such that, given oracle access to $\mathtt{A}$, the following holds.*

$$\Pr\left[\left\langle \mathtt{A}_2(\mathsf{pp}, \mathsf{st}, \mathsf{x}), \mathtt{V}(\mathsf{pp}, \mathsf{x}) \right\rangle = 1 \wedge (\mathsf{x}, \mathsf{w}) \notin \mathsf{R} \;\middle|\; \begin{array}{r} \mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda) \\ (\mathsf{st}, \mathsf{x}) \leftarrow \mathtt{A}_1(\mathsf{pp}) \\ \mathsf{w} \leftarrow \mathtt{E}^{\mathtt{A}}(\mathsf{pp}, \mathsf{x}) \end{array}\right] \leq \mathsf{negl}(\lambda)$$

*$\Pi$ is called honest verifier zero-knowledge (HVZK) if the following holds.*

– ***HVZK****: For every PPT adversary* $\mathtt{A} = (\mathtt{A}_1, \mathtt{A}_2)$*, there exists a PPT simulator* $\mathtt{S}$ *such that the following holds, where* $\mathtt{View}$ *outputs the verifier's view.*

$$\left| \Pr\left[ \mathtt{A}_2(\mathsf{pp}, \mathsf{view}) = 1 \wedge (\mathsf{x}, \mathsf{w}) \in \mathsf{R} \;\middle|\; \begin{array}{r} \mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda) \\ (\mathsf{x}, \mathsf{w}) \leftarrow \mathtt{A}_1(\mathsf{pp}) \\ \mathsf{view} \leftarrow \mathtt{S}(\mathsf{pp}, \mathsf{x}) \end{array} \right] - \Pr\left[ \mathtt{A}_2(\mathsf{pp}, \mathsf{view}) = 1 \wedge (\mathsf{x}, \mathsf{w}) \in \mathsf{R} \;\middle|\; \begin{array}{r} \mathsf{pp} \leftarrow \mathtt{Setup}(1^\lambda) \\ (\mathsf{x}, \mathsf{w}) \leftarrow \mathtt{A}_1(\mathsf{pp}) \\ \mathsf{view} \leftarrow \mathtt{View}\big(\mathtt{P}(\mathsf{pp}, \mathsf{x}, \mathsf{w}), \mathtt{V}(\mathsf{pp}, \mathsf{x})\big) \end{array} \right] \right| \leq \mathsf{negl}(\lambda)$$

*Additionally,* $\Pi$ *is called public coin if all messages from the honest verifier can be computed as a deterministic function of a random public input.*

If an interactive argument of knowledge is public-coin, it can be transformed into a non-interactive version using the Fiat-Shamir transform [FS86]. Furthermore, if it satisfies the HVZK property, the resulting non-interactive argument becomes a non-interactive zero-knowledge argument.

Next, we review the definition of a polynomial interactive oracle proof (PIOP), which is a special class of interactive oracle proof [BSCS16]. In the following definition, we restrict the polynomials to be univariate, but it can be generalized to the multivariate case, as defined in [CBBZ23].

**Definition 2 (Polynomial Interactive Oracle Proof).** *Let* $\Pi = (\mathtt{Setup}, \mathtt{P}, \mathtt{V})$ *be an interactive oracle proof for a relation* $\mathsf{R}$*, and let* $\mathbb{F}$ *be a finite field.* $\Pi$ *is called a polynomial interactive oracle proof (PIOP), which satisfies the following.*

– *Every message from the prover is a polynomial oracle* $([\![\boldsymbol{f}]\!], d)$*, where* $\boldsymbol{f} \in \mathbb{F}[X]$ *of degree* $< d$*.*
– *Every message from the verifier consists of a random challenge in* $\mathbb{F}$*.*
– *At the end of the protocol, the verifier receives oracle access to polynomial evaluations at any points.*

$\Pi$ *is called a honest verifier zero-knowledge PIOP if it satisfies the HVZK property, where* $\mathtt{View}$ *outputs the messages from the verifier and the responses to polynomial evaluation queries. The complexity of PIOP is measured as follows:*

– ***Prover complexity****: The total runtime of the PIOP prover.*
– ***Verifier complexity****: The total runtime of the PIOP verifier.*
– ***Query complexity****: The number of queries made by the verifier to the polynomial oracles.*
– ***Size of proof oracles****: The total length of the transmitted polynomials.*
– ***Size of the witness****: The total length of the witness polynomials.*

In [CHM+20], Chiesa et al. formalize how a PIOP can be compiled into an argument of knowledge using a polynomial commitment scheme (PCS). In brief, the compilation replaces all oracle polynomials in the PIOP with PCS commitments and appends evaluation proofs from the PCS for each polynomial query in the PIOP. For further details, we refer to Appendix A.

### 2.3 Multiparty Homomorphic Encryption

Multiparty homomorphic encryption (MPHE) is a variant of homomorphic encryption that enables homomorphic computation over encrypted data from mutually untrusting parties. The main challenge in constructing MPHE lies in designing an efficient distributed key generation protocol, where parties jointly generate a public key for encryption and homomorphic operations. In this work, we specifically employ the multiparty variant of the BFV scheme [Bra12, FV12] from [KLSW24], as it achieves optimal round complexity for establishing joint public keys. Below, we present its algorithms for setup, key generation, encryption, and decryption based on the residue number system (RNS) implementation [HPS19]. For the correctness of each algorithm, we refer to [KLSW24].

- $\underline{\texttt{Setup}}(1^\lambda, N) \to \mathsf{pp}$: Given a security parameter $\lambda$ and a ring degree $N$, choose a ciphertext modulus $q = \prod_{i=0}^{\ell-1} q_i$, where the $q_i$'s are distinct prime numbers, a plaintext modulus $t$, a key distribution $\chi$ over $R$, a noise distribution $\psi$ over $R$, a noise bound $B_{\mathtt{DD}}$ for distributed decryption, and generate common random strings $\boldsymbol{u}_{\mathsf{ek}} \leftarrow \mathcal{U}(R_q)$, $\vec{\boldsymbol{u}}_{\mathsf{rlk}} \leftarrow \mathcal{U}(R_q^{2\ell})$, and $\vec{\boldsymbol{u}}_{\mathsf{atk}} \leftarrow \mathcal{U}(R_q^{2\ell})$. Return the public parameter $\mathsf{pp} = (R, q, t, \chi, \psi, B_{\mathtt{DD}}, \boldsymbol{u}_{\mathsf{ek}}, \vec{\boldsymbol{u}}_{\mathsf{rlk}}, \vec{\boldsymbol{u}}_{\mathsf{atk}})$.
- $\underline{\texttt{DistKeyGen}}(\mathsf{pp}) \to (\mathsf{dsk}, \mathsf{dpk})$: Given a public parameter $\mathsf{pp}$, generate a secret key share $\mathsf{dsk}$ and a public key share $\mathsf{dpk} = (\mathsf{dek}, \mathsf{drlk}, \mathsf{datk})$ as follows.
  1. Sample $\boldsymbol{s} \leftarrow \chi$, and set the secret key share as $\mathsf{dsk} = \boldsymbol{s} \in R_q$.
  2. Sample $\boldsymbol{e}_{\mathsf{ek}} \leftarrow \psi$, compute $\boldsymbol{p} = -\boldsymbol{u}_{\mathsf{ek}}\boldsymbol{s} + \boldsymbol{e}_{\mathsf{ek}} \pmod q$, and set the encryption key share as $\mathsf{dek}^{(i)} = \boldsymbol{p} \in R_q$.
  3. Sample $\boldsymbol{f}_{\mathsf{rlk}} \leftarrow \chi$, $\vec{\boldsymbol{e}}_{\mathsf{rlk}} \leftarrow \psi^{3\ell}$, and compute the following for $\vec{\boldsymbol{u}}_{\mathsf{rlk}} = \vec{\boldsymbol{u}}_{\mathsf{rlk},0} \| \vec{\boldsymbol{u}}_{\mathsf{rlk},1}$, $\vec{\boldsymbol{e}}_{\mathsf{rlk}} = \vec{\boldsymbol{e}}_{\mathsf{rlk},0} \| \vec{\boldsymbol{e}}_{\mathsf{rlk},1} \| \vec{\boldsymbol{e}}_{\mathsf{rlk},2}$, and $\vec{g} = (q/q_0, \ldots, q/q_{\ell-1})$.
$$\begin{aligned}
\vec{\boldsymbol{r}}_0 &= -\boldsymbol{s} \cdot \vec{\boldsymbol{u}}_{\mathsf{rlk},0} + \vec{\boldsymbol{e}}_{\mathsf{rlk},0} \pmod q \\
\vec{\boldsymbol{r}}_1 &= -\boldsymbol{f}_{\mathsf{rlk}} \cdot \vec{\boldsymbol{u}}_{\mathsf{rlk},0} + \boldsymbol{s} \cdot \vec{g} + \vec{\boldsymbol{e}}_{\mathsf{rlk},1} \pmod q \\
\vec{\boldsymbol{r}}_2 &= -\boldsymbol{s} \cdot \vec{\boldsymbol{u}}_{\mathsf{rlk},1} - \boldsymbol{f}_{\mathsf{rlk}} \cdot \vec{g} + \vec{\boldsymbol{e}}_{\mathsf{rlk},2} \pmod q
\end{aligned}$$
     Set the relinearization key share as $\mathsf{drlk} = (\vec{\boldsymbol{r}}_0, \vec{\boldsymbol{r}}_1, \vec{\boldsymbol{r}}_2) \in R_q^{3\ell}$.
  4. Sample $\vec{\boldsymbol{e}}_{\mathsf{atk}} \leftarrow \psi^{2\ell}$, and compute the following for $\vec{\boldsymbol{u}}_{\mathsf{atk}} = \vec{\boldsymbol{u}}_{\mathsf{atk},0} \| \vec{\boldsymbol{u}}_{\mathsf{atk},1}$, $\vec{\boldsymbol{e}}_{\mathsf{atk}} = \vec{\boldsymbol{e}}_{\mathsf{atk},0} \| \vec{\boldsymbol{e}}_{\mathsf{atk},1}$, and $\vec{g} = (q/q_0, \ldots, q/q_{\ell-1})$.
$$\begin{aligned}
\vec{\boldsymbol{a}}_0 &= -\boldsymbol{s} \cdot \vec{\boldsymbol{u}}_{\mathsf{atk},0} + \boldsymbol{s}(X^{-1}) \cdot \vec{g} + \vec{\boldsymbol{e}}_{\mathsf{atk},0} \pmod q \\
\vec{\boldsymbol{a}}_1 &= -\boldsymbol{s} \cdot \vec{\boldsymbol{u}}_{\mathsf{atk},1} + \boldsymbol{s}(X^{5}) \cdot \vec{g} + \vec{\boldsymbol{e}}_{\mathsf{atk},1} \pmod q
\end{aligned}$$
     Set the automorphism key share as $\mathsf{datk} = (\vec{\boldsymbol{a}}_0, \vec{\boldsymbol{a}}_1) \in R_q^{2\ell}$.[3]
- $\underline{\texttt{JointKeyGen}}(\{\mathsf{dpk}^{(i)}\}_{i \in I}) \to \mathsf{pk}$: Given a collection of public key shares $\{\mathsf{dpk}^{(i)} = (\mathsf{dek}^{(i)}, \mathsf{drlk}^{(i)}, \mathsf{datk}^{(i)})\}_{i \in I}$ for an index set $I$, compute
$$\mathsf{ek} = \sum_{i \in I} \mathsf{dek}^{(i)}, \quad \mathsf{rlk} = \sum_{i \in I} \mathsf{drlk}^{(i)}, \quad \mathsf{atk} = \sum_{i \in I} \mathsf{datk}^{(i)} \pmod q,$$
  and return the joint public key $\mathsf{pk} = (\mathsf{ek}, \mathsf{rlk}, \mathsf{atk})$.

[3] Here, we consider only the automorphisms $X \mapsto X^{-1}$ and $X \mapsto X^5$ for simplicity, as other automorphisms can be performed through their combinations.

– $\underline{\mathtt{Enc}}(\mathsf{ek}, \boldsymbol{m}) \to \mathsf{ct}$: Given a joint encryption key $\mathsf{ek} = \boldsymbol{p}$ and a plaintext $\boldsymbol{m} \in R_t$, sample $\boldsymbol{e}_0, \boldsymbol{e}_1 \leftarrow \psi$, $\boldsymbol{f} \leftarrow \chi$, and return

$$\mathsf{ct} = \boldsymbol{f} \cdot (\boldsymbol{p}, \boldsymbol{u}_{\mathsf{ek}}) + (\lfloor q/t \rceil \cdot \boldsymbol{m} + \boldsymbol{e}_0, \boldsymbol{e}_1) \pmod{q}.$$

– $\underline{\mathtt{Eval}}(\mathcal{C}, \mathsf{rlk}, \mathsf{atk}, \mathsf{ct}_0, \ldots, \mathsf{ct}_{k-1}) \to \mathsf{ct}$: Given an arithmetic circuit $\mathcal{C}$ over $R_t$ consisting of additions, multiplications, and automorphisms, a joint public key $\mathsf{pk}$ and input ciphertexts $\mathsf{ct}_0, \ldots, \mathsf{ct}_{k-1} \in R_q^2$, perform homomorphic operations according to the circuit $\mathcal{C}$, and return the output ciphertext $\mathsf{ct} \in R_q^2$.
– $\underline{\mathtt{DistDec}}(\mathsf{dsk}, \mathsf{ct}) \to \mathsf{dct}$: Given a ciphertext $\mathsf{ct} = (\boldsymbol{c}_0, \boldsymbol{c}_1)$ and a secret key share $\mathsf{dsk} = \boldsymbol{s}$, sample $\boldsymbol{e} \leftarrow \mathcal{U}(R_{B_{\mathtt{DD}}})$, compute $\boldsymbol{d} = \boldsymbol{c}_1 \boldsymbol{s} + \boldsymbol{e} \pmod{q}$, and return the partial decryption $\mathsf{dct} = \boldsymbol{d}$.
– $\underline{\mathtt{JointDec}}(\mathsf{ct}, \{\mathsf{dct}^{(i)}\}_{i \in I}) \to \boldsymbol{m}$: Given a ciphertext $\mathsf{ct} = (\boldsymbol{c}_0, \boldsymbol{c}_1)$ and a collection of partial decryptions from all parties in $I$, return the output plaintext $\boldsymbol{m} = \left\lfloor \frac{t}{q} \left( \boldsymbol{c}_0 + \sum_{i \in I} \mathsf{dct}^{(i)} \right) \right\rceil \pmod{t}$.

An MPHE scheme can be used to construct a general MPC protocol with constant round complexity independent of the target circuit and low communication cost, which depends only on the input and output sizes. The basic pipeline of an MPHE-based MPC protocol consists of the setup, encryption, evaluation, and decryption phases, which operate as follows.

**Setup Phase**

1. For each $i \in I$, the $i$-th party generates a key share $(\mathsf{dsk}^{(i)}, \mathsf{dpk}^{(i)}) \leftarrow \mathtt{DistKeyGen}(\mathsf{pp})$, and broadcasts the public key share $\mathsf{dpk}^{(i)}$ to the other parties.
2. Each party locally computes the joint public key $\mathsf{pk} = (\mathsf{ek}, \mathsf{rlk}, \mathsf{atk}) \leftarrow \mathtt{JointKeyGen}(\{\mathsf{dpk}^{(i)}\}_{i \in I})$ by aggregating the public key shares from all parties.

**Encryption Phase**[a]

1. For each $i \in I$, the $i$-th party generates a ciphertext $\mathsf{ct}^{(i)} \leftarrow \mathtt{Enc}(\mathsf{ek}, \boldsymbol{m}^{(i)})$ using its input $\boldsymbol{m}^{(i)}$ and the joint encryption key $\mathsf{ek}$, and broadcasts it to the other parties.

**Evaluation Phase**

1. Each party locally computes an output ciphertext $\mathsf{ct} \leftarrow \mathtt{Eval}(\mathcal{C}, \mathsf{rlk}, \mathsf{atk}, \{\mathsf{ct}^{(i)}\}_{i \in I})$ using input ciphertexts from all parties and the joint evaluation keys $\mathsf{rlk}$ and $\mathsf{atk}$.

**Decryption Phase**

1. For each $i \in I$, the $i$-th party generates a partial decryption $\mathsf{dct}^{(i)} \leftarrow \mathtt{DistDec}(\mathsf{dsk}^{(i)}, \mathsf{ct})$ with its secret key share $\mathsf{dsk}^{(i)}$, and broadcast the partial decryption to the other parties.
2. Each party locally computes the output $\boldsymbol{m} \leftarrow \mathtt{JointDec}(\mathsf{ct}, \{\mathsf{dct}^{(i)}\}_{i \in I})$ by aggregating the partial decryptions from all parties.

[a] For simplicity, we assume the case where each party sends one ciphertext, but this can be easily extended to more general settings.

Fig. 1: Overall pipeline for an MPHE-based MPC protocol.

The security of the above protocol can be proven in the semi-honest setting, assuming the hardness of the RLWE problem [LPR10] with the circular security assumption [MV24] and in the common random string (CRS) model as analyzed in [KLSW24]. To enhance the security of the above protocol to malicious security, the general compilation method in [AJLA+12] can be utilized. In the compilation method in [AJLA+12], each party additionally generates a zero-knowledge argument of knowledge for their own MPHE ciphertexts and public keys at the setup, encryption, and decryption phases. We note that the proof for the evaluation phase is not required, as each party performs homomorphic operations by itself, and the correctness of homomorphic operations is ensured if verification in the setup and encryption phases is passed. Hence, a passively secure MPHE-based MPC protocol can be efficiently transformed into a maliciously secure one at the cost of proof generation. For further details on the compilation method, we refer to [AJLA+12].

## 3 Review of Univariate PIOP

In this section, we review two univariate PIOPs from [BSCR+19]: the row check PIOP and the linear check PIOP. We observe that these PIOPs can be utilized to construct PIOPs for MPHE, although they were originally proposed for verifying the satisfiability of rank-1 constraint systems (R1CS). In the remainder of this section, we review the details of the row check PIOP and the linear check PIOP. In particular, we focus on the case where the length of witness vectors equals the ring dimension $N$, and the base finite field $\mathbb{F}$ is a prime field $\mathbb{Z}_p$ satisfying $2N \mid p-1$. All proofs are deferred to Appendix C.

### 3.1 Polynomial Encoding

To begin with, we review the polynomial encoding method from [BSCR+19], which encodes vectors in $\mathbb{Z}_p^N$ as polynomials in $\mathbb{Z}_p[X]$. Based on the polynomial encoding method, the row check PIOP and the linear check PIOP can prove relations between witness vectors in $\mathbb{Z}_p^N$ through polynomial evaluations. Let $\mathbb{H} = \{h_0, \ldots, h_{N-1}\} \subseteq \mathbb{Z}_p^*$ be a multiplicative subgroup of order $N$. We note that such a subgroup always exists since $2N \mid p-1$. We define the polynomial encoding and its randomized version as follows.

- $\underline{\texttt{Ecd}}(\vec{w}) \to \boldsymbol{w}$: Given a vector $\vec{w} \in \mathbb{Z}_p^N$, output the polynomial $\boldsymbol{w} \in \mathbb{Z}_p^{<N}[X]$ such that $\boldsymbol{w}(h_i) = w_i$ for $0 \le i < N$.
- $\underline{\texttt{REcd}}(\vec{w}) \to \hat{\boldsymbol{w}}$: Given a vector $\vec{w} \in \mathbb{Z}_p^N$, uniformly sample a polynomial $\hat{\boldsymbol{w}} \in \mathbb{Z}_p^{<L}[X]$ such that $\hat{\boldsymbol{w}}(h_i) = w_i$ for $0 \le i < N$.

For the polynomial $\boldsymbol{w}$, it is uniquely determined by the degree upper bound. However, directly evaluating $\boldsymbol{w}$ can leak information about $\vec{w}$, potentially compromising zero-knowledgeness. To address this, we utilize a randomized version

of polynomial encoding. Specifically, for the polynomial $\hat{\boldsymbol{w}}$, there are $L-N$ possible choices. If we sample $\hat{\boldsymbol{w}}$ uniformly, the evaluation results $\hat{\boldsymbol{w}}(\alpha)$ for $\alpha \notin \mathbb{H}$ remain independent of $\vec{w}$ after up to $L-N$ evaluations, a property known as bounded independence. The HVZK properties of both the row check PIOP and the linear check PIOP depend on this property. We note that both $\texttt{Ecd}$ and $\texttt{REcd}$ can be processed with $O(N \log N)$ time complexity using the number-theoretic transform (NTT). For further details on polynomial encoding, we refer to [BSCR+19].

### 3.2 Row Check PIOP

The row check PIOP is used to prove arithmetic relations between witness vectors in $\mathbb{Z}_p^N$. More precisely, it allows proving the satisfiability of multivariate polynomials over the product ring $\mathbb{Z}_p^N$, where each coefficient is a vector in $\mathbb{Z}_p^N$, and addition and multiplication are performed component-wise. Let $\boldsymbol{z}_{\mathbb{H}} = \prod_{i=0}^{N-1}(X - h_i) = X^N - 1$ be the vanishing polynomial over $\mathbb{H}$, and let $\vec{\mathtt{C}} \in (\mathbb{Z}_p^N)[\vec{X}_0, \ldots, \vec{X}_{n-1}]$ be a $n$-ary polynomial over $\mathbb{Z}_p^N$, whose satisfiability we aim to prove. By utilizing polynomial encoding, its satisfiability can be reformulated as follows, where $\mathbf{C} \in (\mathbb{Z}_p[X])[\boldsymbol{X}_0, \ldots, \boldsymbol{X}_{n-1}]$ is an $n$-ary polynomial obtained by applying $\texttt{Ecd}$ to each coefficient of $\vec{\mathtt{C}}$, and $\boldsymbol{a}_i = \texttt{Ecd}(\vec{a}_i)$ for $0 \le i < k$.

$$\vec{\mathtt{C}}(\vec{a}_0, \ldots, \vec{a}_{k-1}) = \vec{0} \iff \forall h_i \in \mathbb{H},\ \mathbf{C}(\boldsymbol{a}_0, \ldots, \boldsymbol{a}_{k-1})(h_i) = 0$$

The above equivalence holds because $\mathbf{C}(\boldsymbol{a}_0, \ldots, \boldsymbol{a}_{k-1})(h_i)$ is equal to the $i$-th component of $\vec{\mathtt{C}}(\vec{a}_0, \ldots, \vec{a}_{k-1})$. Finally, by leveraging the property of the vanishing polynomial, the latter condition can be reformulated as follows.

$$\forall h_i \in \mathbb{H},\ \mathbf{C}(\boldsymbol{a}_0, \ldots, \boldsymbol{a}_{k-1})(h_i) = 0 \iff \boldsymbol{z}_{\mathbb{H}} \mid \mathbf{C}(\boldsymbol{a}_0, \ldots, \boldsymbol{a}_{k-1})$$

Hence, it suffices to verify the final divisibility condition, which can be efficiently checked through polynomial evaluations using the Schwartz-Zippel lemma. Below, we provide a detailed description of the row check PIOP, denoted by $\Pi_{\texttt{RC}}$.

**Theorem 1 ([BSCR+19, Definition 4.9]).** *Let $\vec{\mathtt{C}}$ be a $n$-ary polynomial over $\mathbb{Z}_p^N$ with degree $d$, $m$ be the number of non-zero terms of $\vec{\mathtt{C}}$, and let $\hat{\boldsymbol{a}}_i \leftarrow \texttt{REcd}(\vec{a}_i)$ for $0 \le i < n$, where $\vec{\mathtt{C}}(\vec{a}_0, \ldots, \vec{a}_{n-1}) = \vec{0}$. Then, $\Pi_{\texttt{RC}}$ in Fig. 7 is an HVZK PIOP with the following complexity.*

- *The soundness error is $O\left(\frac{dN}{p}\right)$.*
- *The prover time is $O((n+m)N \log N + mdN + dN \log(dN))$.*
- *The verifier time is $O(mN \log N + dm)$.*
- *The query complexity is $n+1$.*
- *The size of proof oracles is $\le dL$ elements in $\mathbb{Z}_p$.*
- *The size of witness is $nN$ elements in $Z_p$.*

### 3.3 Linear Check PIOP

The linear check PIOP is used to prove the linear relations between input witness vectors, which can be described as a matrix-vector multiplication for some matrix in $\mathbb{Z}_p^{N\times N}$. It is based on the univariate sumcheck PIOP in [BSCR+19], which enables us to prove that $\sum_{h_i\in\mathbb{H}} \boldsymbol{f}(h_i) = 0$ for some witness polynomial $\boldsymbol{f}$. To utilize the univariate sumcheck PIOP, the satisfiability of matrix-vector multiplication is rephrased as follows. Let $\vec{a}, \vec{b} \in \mathbb{Z}_p^N$ and let $M \in \mathbb{Z}_p^{N\times N}$. Then, by the Schwartz-Zippel lemma, $\vec{b} = M\vec{a}$ holds with probability $1-N/p$ if $\langle \vec{b}-M\vec{a}, \vec{v}\rangle = 0$ holds for a random vector $\vec{v} = (1, v, \ldots, v^{N-1})$. Therefore, assuming that $N/p$ is negligible, it suffices to check the random inner product evaluation to verify the satisfiability of the matrix-vector multiplication.

Using the polynomial encoding method, the inner product evaluation can be reformulated as follows,

$$\begin{aligned}\langle \vec{b} - M\vec{a}, \vec{v}\rangle = 0 &\iff \langle \vec{b}, \vec{v}\rangle - \langle \vec{a}, M^\top \vec{v}\rangle = 0 \\ &\iff \sum_{h_i\in\mathbb{H}} (\boldsymbol{b}\cdot\boldsymbol{v} - \boldsymbol{a}\cdot\boldsymbol{w})(h_i) = 0\end{aligned}$$

where $\boldsymbol{a} = \mathtt{Ecd}(\vec{a})$, $\boldsymbol{b} = \mathtt{Ecd}(\vec{b})$, $\boldsymbol{v} = \mathtt{Ecd}(\vec{v})$, and $\boldsymbol{w} = \mathtt{Ecd}(M^\top\vec{v})$. Hence, by applying the univariate sumcheck protocol to the polynomial $(\boldsymbol{b}\cdot\boldsymbol{v} - \boldsymbol{a}\cdot\boldsymbol{w})$, we can prove the matrix-vector multiplication. Below, we present a detailed description of the linear check PIOP, denoted by $\Pi_{\mathtt{LC}}$, which enables proving multiple relations in a batched manner.

**Theorem 2 ([BSCR+19, Theorem 6.2]).** *Let $k$ be the number of non-zero entries in the input matrix $M \in \mathbb{Z}_p^{N\times N}$, and let $\hat{\boldsymbol{a}}_i \leftarrow \mathtt{REcd}(\vec{a}_i)$, $\hat{\boldsymbol{b}}_i \leftarrow \mathtt{REcd}(\vec{b}_i)$ for $0 \le i < n$, where $\vec{b}_i = M\vec{a}_i$. Then, $\Pi_{\mathtt{LC}}$ in Fig. 8 is an HVZK PIOP with the following complexity.*

- *The soundness error is $O(\frac{N+n}{p-N})$.*
- *The prover time is $O(k + nN\log N)$.*
- *The verifier time is $O(n + k + N\log N)$.*
- *The query complexity is $2n + 3$.*
- *The size of the proof oracles is $\le 2L + 2N$ elements in $\mathbb{Z}_p$.*
- *The size of witness is $2nN$ elements in $\mathbb{Z}_p$.*

## 4 PIOP for MPHE

In this section, we present a novel approach for designing zero-knowledge arguments of knowledge (ZKAoKs) for MPHE based on the PIOP framework. ZKAoKs for MPHE are crucial for achieving malicious security in MPHE-based MPC protocols. As shown in [AJLA+12], passively secure MPHE-based MPC protocols can be compiled into maliciously secure ones by integrating ZKAoKs that prevent deviations from honest protocol execution. Specifically, these ZKAoKs

are required to validate the well-formedness of public key shares, input ciphertexts, and partial decryption shares exchanged during the setup, encryption, and decryption phases. We note that the evaluation phase does not require ZKAoKs for validation, as no communication occurs during this phase, and the validity of input and output ciphertexts is checked in the encryption and decryption phases, respectively. Thus, the design of ZKAoKs for MPHE primarily focuses on the setup, encryption, and decryption phases.

To construct efficient ZKAoKs for MPHE, we make a novel observation that univariate PIOPs, originally developed for SNARK constructions, can be effectively adapted to this setting. To the best of our knowledge, this approach has not been explored before, primarily due to inconsistencies in witness spaces. Specifically, ZKAoKs for MPHE typically require proving constraints over a polynomial ring $R_q$, which serves as the ciphertext space. In contrast, PIOPs are designed for proving constraints over a product ring $\mathbb{Z}_p^N$, which is widely used in SNARKs to represent general-purpose arithmetic circuits.

In the remainder of this section, we first address how we resolve these inconsistency issues. In short, we propose a novel approach that converts MPHE constraints defined over $R_q$ into constraints over $\mathbb{Z}_p^N$, enabling efficient verification via PIOPs. This conversion bridges the gap between ZKAoKs for MPHE and PIOPs, allowing us to leverage the expressiveness of PIOPs. After establishing this connection, we detail how the constraints required for each phase of the MPHE-based MPC protocol can be efficiently represented in PIOP form.

### 4.1 Conversion of Constraints from $R_q$ to $\mathbb{Z}_p^N$

To begin, we show how constraints over the polynomial ring $R_q$ can be translated into equivalent constraints over the product ring $\mathbb{Z}_p^N$, which can be verified via row check PIOPs and linear check PIOPs in Section 3. To address the inconsistencies between $R_q$ and $\mathbb{Z}_p^N$, we need to resolve two key issues: the discrepancy in moduli $q$ and $p$, and the structural difference between a polynomial ring and a product ring.

We overcome these challenges by incorporating two optimization techniques: modulus switching and vector representations of polynomials. The modulus switching operation is a homomorphic operation that enables the ciphertext modulus to be changed while preserving the ciphertext structure. By using this operation, we translate constraints over $R_q$ to $R_p$ by switching the modulus from $q$ to $p$, thus resolving the modulus inconsistency issue. On the other hand, we utilize two vector representations of polynomials, coefficient form and number-theoretic transform (NTT) form, to translate constraints over $R_p$ to $\mathbb{Z}_p^N$. In particular, the NTT is an isomorphism between $R_p$ and $\mathbb{Z}_p^N$, which allows arithmetic over $R_p$ to be rephrased as arithmetic over $\mathbb{Z}_p^N$, since the isomorphism preserves algebraic structures. This enables us to translate arithmetic constraints over $R_p$ into $\mathbb{Z}_p^N$, which can be verified through univariate PIOPs. Thus, by combining modulus switching and vector representations, we successfully address the inconsistency issues in witness spaces. In the following, we describe in detail how we utilize modulus switching and vector representations for our purposes.

**Modulus Switching.** As reviewed in Section 2.3, the ciphertext modulus $q$ is typically chosen as a product of distinct primes. However, the PIOPs described in Section 3 are designed to support vectors over a prime field, whereas $q$ itself is not prime. One possible approach to address this issue is to prove constraints separately for each $R_{q_i}$ by leveraging the isomorphism $R_q \cong \prod_{i=0}^{\ell-1} R_{q_i}$. However, this approach significantly degrades performance in the PIOP setting, as each $q_i$ is too small to achieve a negligible soundness error, and we would need to repeat the proof generation process $\ell$ times, leading to inefficiencies in both soundness and computational cost. Instead of working directly with the modulus $q$, we incorporate a modulus switching operation to convert $q$ into a prime modulus $p$, where $p \approx q$. This approach enables us to achieve both negligible soundness error and efficient proof generation by working over the prime modulus $p$.

In short, modulus switching is a simple rescaling operation that maps a ring element $\boldsymbol{a} \in R_p$ to $\left\lfloor \frac{q}{p}\boldsymbol{a} \right\rceil \in R_q$, or vice versa. The key advantage of modulus switching is that it preserves the validity of public key shares and ciphertexts, even when they are initially generated under modulus $p$ and later switched to modulus $q$. To leverage this property, we modify the MPHE-based MPC protocol in Fig. 1. Specifically, we generate public key shares, ciphertexts, and partial decryptions in $R_p$ for efficient proof generation and switch the modulus from $p$ to $q$ only during the evaluation phase to perform homomorphic operations, as described below.

**Modified Evaluation Phase**

1. Each party locally computes $\mathsf{ct}' \leftarrow \mathtt{Eval}\Big(\mathcal{C}, \left\lfloor \frac{q}{p}\mathsf{rlk} \right\rceil, \left\lfloor \frac{q}{p}\mathsf{atk} \right\rceil, \{\left\lfloor \frac{q}{p}\mathsf{ct}^{(i)} \right\rceil\}_{i\in I}\Big)$ using input ciphertexts $\mathsf{ct}^{(i)} \in R_p$ from all parties and the joint evaluation keys $\mathsf{rlk} \in R_p^{3\ell}$ and $\mathsf{atk} \in R_p^{2\ell}$. The output ciphertext is then set as $\mathsf{ct} = \left\lfloor \frac{p}{q}\mathsf{ct}' \right\rceil \in R_p^2$.

Fig. 2: Modified evaluation phase of the MPHE-based MPC protocol

We note that the correctness of the modified evaluation phase is ensured if $\left\lfloor \frac{q}{p}\mathsf{rlk} \right\rceil$, $\left\lfloor \frac{q}{p}\mathsf{atk} \right\rceil$, and $\{\left\lfloor \frac{q}{p}\mathsf{ct}^{(i)} \right\rceil\}_{i\in I}$ are valid evaluation keys and ciphertexts, which can alternatively be proven by verifying the validity of $\mathsf{rlk}, \mathsf{atk}$, and $\{\mathsf{ct}^{(i)}\}_{i\in I}$ in the space of $R_p$. Thus, thanks to modulus switching, we can now focus on constraints over $R_p$ instead of $R_q$.

**Vector Representation.** To prove the validity of MPHE public key shares and ciphertexts, we primarily need to verify two types of constraints over the polynomial ring $R_p$: the arithmetic relations between polynomials and the boundedness of polynomial coefficients. However, as noted earlier, we need to convert these constraints into constraints over a product ring $\mathbb{Z}_p^N$ to leverage the PIOPs in Section 3. To do so, we use two types of vector representations for polynomials: the coefficient form and the number-theoretic transform (NTT) form, which are defined as follows.

– $\underline{\texttt{Coeff}}(\boldsymbol{a}) \to \vec{a}$: Given a polynomial $\boldsymbol{a} = \sum_{i=0}^{N-1} a_i X^i \in R_p$, outputs a vector $\vec{a} = (a_0, \ldots, a_{N-1}) \in \mathbb{Z}_p^N$.
– $\underline{\texttt{NTT}}(\boldsymbol{a}) \to \underline{\vec{a}}$: Given a polynomial $\boldsymbol{a} = \sum_{i=0}^{N-1} a_i X^i \in R_p$, outputs a vector $\underline{\vec{a}} = \big(\boldsymbol{a}(\xi), \boldsymbol{a}(\xi^3), \ldots, \boldsymbol{a}(\xi^{2N-1})\big) \in \mathbb{Z}_p^N$, where $\xi$ is a $2N$-th root of unity modulo $p$.

The coefficient form is simply a vector of coefficients, which preserves the norm but not the algebraic structure. In contrast, the NTT form preserves the algebraic structure by exploiting the isomorphism $R_p \cong \mathbb{Z}_p^N$ when $\mathbb{Z}_p$ has a $2N$-th root of unity, i.e., $2N \mid p-1$. Computing the NTT form requires $O(N \log N)$ time complexity and preserves the algebraic structure due to the isomorphism, while the norm is not preserved. Therefore, by utilizing both vector representations as witnesses, we can translate all the required constraints for MPHE over $R_p$ into constraints over $\mathbb{Z}_p^N$.

### 4.2 PIOPs for Vector Representations

Since both arithmetic relations and the boundedness of coefficients need to be proven, it is essential to use both vector representations when constructing PIOPs for MPHE. This inherently requires two PIOPs for vector representations: one for the NTT operation and another for the $L^\infty$-norm bound. The former verifies the relationship between the coefficient form and the NTT form, while the latter ensures the boundedness of the coefficient form.

**PIOP for NTT.** For the PIOP for NTT, we observe that the NTT operation can be represented as a matrix-vector multiplication using a Vandermonde matrix. Specifically, for the coefficient form $\vec{a} \in \mathbb{Z}_p^N$ and its NTT form $\underline{\vec{a}}$ corresponding to the polynomial $\boldsymbol{a} \in R_p$, the relation $\underline{\vec{a}} = T\vec{a}$ holds, where $T \in \mathbb{Z}_p^{N \times N}$ is the Vandermonde matrix defined as follows. This suggests that we can apply a linear check PIOP to verify this relation.

$$T = \begin{bmatrix} 1 & \xi & \cdots & \xi^{N-1} \\ 1 & \xi^3 & \cdots & \xi^{3(N-1)} \\ \vdots & \vdots & & \vdots \\ 1 & \xi^{2N-1} & \cdots & \xi^{(2N-1)(N-1)} \end{bmatrix}$$

However, a direct application results in $O(N^2)$ complexity for both the prover and the verifier, as the complexity is determined by the number of nonzero entries in the target matrix. Since the Vandermonde matrix contains $N^2$ nonzero entries, this leads to quadratic complexity. In particular, this quadratic complexity arises from the computation of $T^\top \vec{v}$ in the linear check PIOP $\Pi_{\texttt{LC}}$ shown in Fig. 8.

To resolve this issue, we devise the following alternative method to compute $T^\top \vec{v}$,

$$T^\top \vec{v} = N \cdot \texttt{iNTT}(J\vec{v})$$

where $J$ is the backward identity matrix and $\texttt{iNTT}$ is the inverse number-theoretic transform. With this optimization, the computation of $T^\top \vec{v}$ achieves $O(N \log N)$

complexity. This is because multiplying $J$ by $\vec{v}$ requires $O(N)$ complexity due to the sparsity of $J$, while $\texttt{iNTT}$ can be computed in $O(N \log N)$ complexity, similar to the $\texttt{NTT}$ operation. As a result, this optimization reduces the complexity of the linear check PIOP for the matrix $T$ from $O(N^2)$ to $O(N \log N)$

**PIOP for $L^\infty$-norm.** Regarding the PIOP for the $L^\infty$-norm bound, we represent the range constraint as an arithmetic circuit over $\mathbb{Z}_p^N$ and employ the row check PIOP for verification. Additionally, we optimize this construction to support batched verification of bounds for multiple witness vectors, as this scenario frequently arises in the context of PIOPs for MPHE.

To be specific, the goal of this PIOP is to prove $\|\vec{a}\|_\infty \leq B$ for a witness vector $\vec{a} \in \mathbb{Z}_p^N$. To prove this, we utilize the ternary representation method from [LNSW13]. For $B > 0$, let $k = \lfloor \log B \rfloor + 1$ and define $B_0 = \lceil \frac{B}{2} \rceil$, $B_1 = \lceil \frac{B-B_0}{2} \rceil$, $B_2 = \lceil \frac{B-B_0-B_1}{2} \rceil, \ldots, B_{k-1} = 1$, and $B_k = 0$. If $\|\vec{a}\|_\infty \leq B$ holds, it can be decomposed as $\vec{a} = \sum_{i=0}^{k-1} B_i \cdot \vec{a}_i$, where each $\vec{a}_i$ is a ternary vector, i.e., each entry is either -1, 0, or 1. This upper bound on the norm can then be translated into the following arithmetic constraints: $\vec{a} - \sum_{i=0}^{k-1} B_i \cdot \vec{a}_i = \vec{0}$ and $\vec{a}_i \circ (\vec{a}_i - \vec{1}) \circ (\vec{a}_i + \vec{1}) = \vec{0}$ for $0 \leq i < k$, where the constraints can be verified using row check PIOPs, as illustrated in Fig. 3. Once the arithmetic constraints for a single witness vector are established, they can be efficiently verified in a batched manner by utilizing the Schwartz-Zippel lemma. Below, we provide an analysis of the norm check PIOP $\Pi_{\texttt{NC}}$.

**Theorem 3 (Norm check).** *Let $\hat{\boldsymbol{a}}_i \leftarrow \texttt{REcd}(\vec{a}_i)$ for $0 \leq i < n$, where $\|\vec{a}_i\|_\infty \leq B$. Then, $\Pi_{\texttt{NC}}$ is an HVZK PIOP with the following complexity, where $k = \lfloor \log B \rfloor + 1$.*

- *The soundness error is $O\left(\frac{N+nk}{p-N}\right)$.*
- *The prover time is $O(nkN \log N)$.*
- *The verifier time is $O(nkN \log N)$.*
- *The query complexity is $nk + n + 1$.*
- *The size of the proof oracles is $(nk + 3)L$ elements in $\mathbb{Z}_p$.*
- *The size of the witness is $nN$ elements in $\mathbb{Z}_p$.*

$$\Pi_{\texttt{NC}}(B; [\![\hat{\boldsymbol{a}}_0]\!], \ldots, [\![\hat{\boldsymbol{a}}_{n-1}]\!])$$

**Instance**: norm bound $B$, and polynomial oracles $([\![\hat{\boldsymbol{a}}_i]\!], L)$ for $0 \leq i < n$.
**Witness**: $\vec{a}_i = \texttt{Dcd}(\hat{\boldsymbol{a}}_i) \in \mathbb{Z}_p^N$ for $0 \leq i < n$.
**Statement**: $\|\vec{a}_i\|_\infty \leq B$ for $0 \leq i < n$.

1. The prover $\texttt{P}$ and the verifier $\texttt{V}$ decompose $B$ into $B_0 = \lceil \frac{B}{2} \rceil$, $B_1 = \lceil \frac{B-B_0}{2} \rceil$, $B_2 = \lceil \frac{B-B_0-B_1}{2} \rceil, \ldots, B_{k-1} = 1$, where $k = \lfloor \log B \rfloor + 1$.
2. The prover $\texttt{P}$ decomposes each $\vec{a}_i$ into $\vec{a}_{i,0}, \ldots, \vec{a}_{i,k-1}$ such that $\vec{a}_i = \sum_{j=0}^{k-1} B_j \cdot \vec{a}_{i,j}$ and $\|\vec{a}_{i,j}\|_\infty \leq 1$, and samples polynomial encodings $\hat{\boldsymbol{a}}_{i,j} \leftarrow \texttt{REcd}(\vec{a}_{i,j})$. Then, $\texttt{P}$ sends polynomial oracles $([\![\hat{\boldsymbol{a}}_{i,j}]\!], L)$ to $\texttt{V}$ for $0 \leq i < n$ and $0 \leq j < k$.
3. The verifier $\texttt{V}$ sends random points $\beta, \gamma \leftarrow \mathcal{U}(\mathbb{Z}_p \setminus \mathbb{H})$.

4. The prover $\mathtt{P}$ and the verifier $\mathtt{V}$ invoke the following PIOPs.

$$\Pi_{\mathtt{RC}}\Bigg(\sum_{i=0}^{n-1}\gamma^i\cdot\Big(\vec{X}_{i(k+1)}-\sum_{j=0}^{k-1}B_j\cdot\vec{X}_{i(k+1)+j+1} + \sum_{j=0}^{k-1}\beta^{j+1}\cdot(\vec{X}^3_{i(k+1)+j+1}-\vec{X}_{i(k+1)+j+1})\Big) ; [\![\boldsymbol{a}_0]\!],[\![\boldsymbol{a}_{0,0}]\!],\ldots,[\![\boldsymbol{a}_{0,k-1}]\!],\ldots,[\![\boldsymbol{a}_{n-1}]\!],[\![\boldsymbol{a}_{n-1,0}]\!],\ldots,[\![\boldsymbol{a}_{n-1,k-1}]\!]\Bigg)$$

Fig. 3: PIOP for $L^\infty$ norm bound check

### 4.3 PIOP for Setup Phase

In the setup phase of Fig. 1, it is essential to prove the validity of the public key shares transmitted during this stage. Once these shares are verified as valid, the joint encryption key, relinearization key, and automorphism key derived from them also remain valid, ensuring the correctness of the subsequent encryption and evaluation phases. Therefore, the primary objective is to establish the validity of the public key shares. As previously discussed, these shares are generated in $R_p$ to enable efficient proof generation through univariate PIOPs. To achieve this, we introduce the following modifications to the setup and key generation algorithms in MPHE.

- $\underline{\mathtt{Setup}}$: Choose a prime modulus such that $2N \mid p-1$ and $p \approx q$, and generate common random strings $\boldsymbol{u}_{\mathsf{ek}} \leftarrow \mathcal{U}(R_p)$, $\vec{\boldsymbol{u}}_{\mathsf{rlk}} \leftarrow \mathcal{U}(R_p^{2\ell})$, and $\vec{\boldsymbol{u}}_{\mathsf{atk}} \leftarrow \mathcal{U}(R_p^{2\ell})$.
- $\underline{\mathtt{DistKeyGen}}$: Generate public key shares $\mathsf{dek} = \boldsymbol{p}$, $\mathsf{drlk} = (\vec{\boldsymbol{r}}_0, \vec{\boldsymbol{r}}_1, \vec{\boldsymbol{r}}_2)$, $\mathsf{datk} = (\vec{\boldsymbol{a}}_0, \vec{\boldsymbol{a}}_1)$ as follows, using the gadget vector $\vec{g}' = (\lfloor p/q_0 \rceil, \ldots, \lfloor p/q_{\ell-1} \rceil)$ instead of $\vec{g} = (q/q_0, \ldots, q/q_{\ell-1})$.

$$\boldsymbol{p} = -\boldsymbol{u}_{\mathsf{ek}}\boldsymbol{s} + \boldsymbol{e}_{\mathsf{ek}} \pmod p \tag{1}$$
$$\vec{\boldsymbol{r}}_0 = -\boldsymbol{s}\cdot\vec{\boldsymbol{u}}_{\mathsf{rlk},0} + \vec{\boldsymbol{e}}_{\mathsf{rlk},0} \pmod p \tag{2}$$
$$\vec{\boldsymbol{r}}_1 = -\boldsymbol{f}_{\mathsf{rlk}}\cdot\vec{\boldsymbol{u}}_{\mathsf{rlk},0} + \boldsymbol{s}\cdot\vec{g}' + \vec{\boldsymbol{e}}_{\mathsf{rlk},1} \pmod p \tag{3}$$
$$\vec{\boldsymbol{r}}_2 = -\boldsymbol{s}\cdot\vec{\boldsymbol{u}}_{\mathsf{rlk},1} - \boldsymbol{f}_{\mathsf{rlk}}\cdot\vec{g}' + \vec{\boldsymbol{e}}_{\mathsf{rlk},2} \pmod p \tag{4}$$
$$\vec{\boldsymbol{a}}_0 = -\boldsymbol{s}\cdot\vec{\boldsymbol{u}}_{\mathsf{atk},0} + \boldsymbol{s}(X^{-1})\cdot\vec{g}' + \vec{\boldsymbol{e}}_{\mathsf{atk},0} \pmod p \tag{5}$$
$$\vec{\boldsymbol{a}}_1 = -\boldsymbol{s}\cdot\vec{\boldsymbol{u}}_{\mathsf{atk},1} + \boldsymbol{s}(X^{5})\cdot\vec{g}' + \vec{\boldsymbol{e}}_{\mathsf{atk},1} \pmod p \tag{6}$$

- $\underline{\mathtt{JointKeyGen}}$: Aggregate the public key shares modulo $p$.

$$\mathsf{ek} = \sum_{i\in I}\mathsf{dek}^{(i)}, \quad \mathsf{rlk} = \sum_{i\in I}\mathsf{drlk}^{(i)}, \quad \mathsf{atk} = \sum_{i\in I}\mathsf{datk}^{(i)} \pmod p,$$

Below, we discuss the validity of each type of public key share, assuming that the $L^\infty$-norm of the samples from the key distribution $\chi$ and the noise distribution $\psi$ is bounded by $B$.

**Encryption Key.** The validity of the joint encryption key $\mathsf{ek}$ is guaranteed if each encryption key share $\mathsf{dek} = \boldsymbol{p}$ is correctly generated, meaning that the witness polynomials $\boldsymbol{s}$ and $\boldsymbol{e}_{\mathsf{ek}}$ satisfy Eq. (1) and their norms are bounded by $B > 0$. These conditions can be efficiently verified using row check PIOPs and norm check PIOPs.

**Relinearization Key.** In the case of the joint relinearization key $\mathsf{rlk}$, we need to verify the validity of $\left\lfloor \frac{q}{p}\mathsf{rlk} \right\rceil$ since homomorphic operations will be performed in $R_q$ during the modified evaluation phase, as described in Fig. 2. The validity of $\left\lfloor \frac{q}{p}\mathsf{rlk} \right\rceil$ is guaranteed if each relinearization key share $\left\lfloor \frac{q}{p}\mathsf{drlk} \right\rceil$ is valid, meaning that the following equation is satisfied for some small-normed polynomials $\boldsymbol{s}$, $\boldsymbol{f}_{\mathsf{rlk}}$, and $\vec{\boldsymbol{e}}'_{\mathsf{rlk}} = \vec{\boldsymbol{e}}'_{\mathsf{rlk},0} \| \vec{\boldsymbol{e}}'_{\mathsf{rlk},1} \| \vec{\boldsymbol{e}}'_{\mathsf{rlk},2}$.

$$\begin{aligned}
\left\lfloor \frac{q}{p}\vec{\boldsymbol{r}}_0 \right\rceil &= -\boldsymbol{s} \cdot \left\lfloor \frac{q}{p}\vec{\boldsymbol{u}}_{\mathsf{rlk},0} \right\rceil + \vec{\boldsymbol{e}}'_{\mathsf{rlk},0} \pmod q \\
\left\lfloor \frac{q}{p}\vec{\boldsymbol{r}}_1 \right\rceil &= -\boldsymbol{f}_{\mathsf{rlk}} \cdot \left\lfloor \frac{q}{p}\vec{\boldsymbol{u}}_{\mathsf{rlk},0} \right\rceil + \boldsymbol{s} \cdot \vec{g} + \vec{\boldsymbol{e}}'_{\mathsf{rlk},1} \pmod q \\
\left\lfloor \frac{q}{p}\vec{\boldsymbol{r}}_2 \right\rceil &= -\boldsymbol{s} \cdot \left\lfloor \frac{q}{p}\vec{\boldsymbol{u}}_{\mathsf{rlk},1} \right\rceil - \boldsymbol{f}_{\mathsf{rlk}} \cdot \vec{g} + \vec{\boldsymbol{e}}'_{\mathsf{rlk},2} \pmod q
\end{aligned}$$

However, thanks to the modulus switching, we obtain the following upper bound for the norm of $\left\|\vec{\boldsymbol{e}}'_{\mathsf{rlk}}\right\|_\infty$.

**Lemma 1.** *For* $\left\lfloor \frac{q}{p}\mathsf{rlk} \right\rceil \in R_q^{3\ell}$, *it holds that* $\left\|\vec{\boldsymbol{e}}'_{\mathsf{rlk}}\right\|_\infty \leq \frac{q}{p}\|\vec{\boldsymbol{e}}_{\mathsf{rlk}}\|_\infty + \frac{1}{2}\left(\frac{q}{p}B + NB + 1\right)$ *if* $\|\boldsymbol{s}\|_\infty \leq B$ *and* $\|\boldsymbol{f}_{\mathsf{rlk}}\|_\infty \leq B$.

Then, we can ensure that $\left\lfloor \frac{q}{p}\mathsf{drlk} \right\rceil$ is a valid relinearization key share in $R_q$ if $\|\vec{\boldsymbol{e}}_{\mathsf{rlk}}\|_\infty$ is small. Therefore, to verify the validity of the relinearization key, it suffices to check the arithmetic relations for $\mathsf{drlk}$, as described in Eqs. (2) to (4), and prove that $\boldsymbol{s}$, $\boldsymbol{f}$, and $\vec{\boldsymbol{e}}_{\mathsf{rlk}}$ have small norms. These checks can be efficiently handled using row check PIOPs and norm check PIOPs.

**Automorphism Key**. For the joint automorphism key $\mathsf{atk}$, the basic rationale is similar to the relinearization case. Let $\vec{\boldsymbol{e}}'_{\mathsf{atk}} = \vec{\boldsymbol{e}}'_{\mathsf{atk},0} \| \vec{\boldsymbol{e}}'_{\mathsf{atk},1} \in R_q^{2\ell}$, which satisfies the following.

$$\begin{aligned}
\left\lfloor \frac{q}{p}\vec{\boldsymbol{a}}_0 \right\rceil &= -\boldsymbol{s} \cdot \left\lfloor \frac{q}{p}\vec{\boldsymbol{u}}_{\mathsf{atk},0} \right\rceil + \boldsymbol{s}(X^{-1}) \cdot \vec{g} + \vec{\boldsymbol{e}}'_{\mathsf{atk},0} \pmod q \\
\left\lfloor \frac{q}{p}\vec{\boldsymbol{a}}_1 \right\rceil &= -\boldsymbol{s} \cdot \left\lfloor \frac{q}{p}\vec{\boldsymbol{u}}_{\mathsf{atk},1} \right\rceil + \boldsymbol{s}(X^{5}) \cdot \vec{g} + \vec{\boldsymbol{e}}_{\mathsf{atk},1} \pmod q
\end{aligned}$$

Then, the following lemma confirms that it suffices to check the satisfiability of Eqs. (5) and (6), and the smallness of $\boldsymbol{s}$ and $\vec{\boldsymbol{e}}_{\mathsf{atk}}$.

**Lemma 2.** *For* $\left\lfloor \frac{q}{p}\mathsf{atk} \right\rceil \in R_q^{2\ell}$, *it holds that* $\left\|\vec{\boldsymbol{e}}'_{\mathsf{atk}}\right\|_\infty \leq \frac{q}{p}\|\vec{\boldsymbol{e}}_{\mathsf{atk}}\|_\infty + \frac{1}{2}\left(\frac{q}{p}B + NB + 1\right)$ *if* $\|\boldsymbol{s}\|_\infty \leq B$.

However, an additional step is required to handle the terms $\boldsymbol{s}(X^{-1})$ and $\boldsymbol{s}(X^5)$ when proving the satisfiability of Eqs. (5) and (6), as we need to prove the relation between these terms and $\boldsymbol{s}$.

To address this issue, we observe that automorphism operations can also be represented as matrix-vector multiplications, which allows us to utilize linear check PIOPs to verify them. More precisely, for an automorphism $\varphi$ of the form $X \mapsto X^k$, the following holds.

$$\mathtt{NTT}(\varphi(\boldsymbol{s})) = \mathtt{NTT}(\boldsymbol{s}(X^k)) = \left(\boldsymbol{s}(\xi^k), \boldsymbol{s}(\xi^{3k}), \ldots, \boldsymbol{s}(\xi^{(2N-1)k})\right)$$

Thus, the components of $\mathtt{NTT}(\varphi(\boldsymbol{s}))$ are simply a permutation of the components of $\mathtt{NTT}(\boldsymbol{s})$, which implies that there exists a permutation matrix $P$ such that $\mathtt{NTT}(\varphi(\boldsymbol{s})) = P \cdot \mathtt{NTT}(\boldsymbol{s})$. Therefore, we can prove the relation between $\varphi(\boldsymbol{s})$ and $\boldsymbol{s}$ using a linear check PIOP for the matrix $P$. Moreover, since $P$ is a sparse matrix, both the prover and verifier complexities remain quasi-linear in $N$. As a result, we can verify the validity of the joint automorphism key through a combination of row check PIOPs, norm check PIOPs, and linear check PIOPs.

Below, we provide complete PIOP descriptions for the validity of public key shares, along with their complexity analysis.

**Theorem 4 (PIOP for Setup Phase).** *$\Pi_{\mathtt{DKG}}$ is an HVZK PIOP with the following complexity, where $k = \lfloor \log B \rfloor + 1$.*

- *The soundness error is $O(\frac{\ell k+N}{p-N})$.*
- *The prover time is $O(\ell k N \log N)$.*
- *The verifier time is $O(\ell k N \log N + \ell^2 \log N)$.*
- *The query complexity is $(5\ell+3)k + 20\ell + 36$.*
- *The size of proof oracles is $\leq (10\ell + 5\ell k + 3k + 20)L + 6N$ elements in $\mathbb{Z}_p$.*
- *The size of witness is $(5\ell+3)N$ elements in $\mathbb{Z}_p$.*

$\Pi_{\mathtt{DKG}}$

**Instance**: $\mathsf{dek} = \boldsymbol{p} \in R_p$, $\mathsf{drlk} = (\vec{\boldsymbol{r}}_0, \vec{\boldsymbol{r}}_1, \vec{\boldsymbol{r}}_2) \in R_p^{3\ell}$, $\mathsf{datk} = (\vec{\boldsymbol{a}}_0, \vec{\boldsymbol{a}}_1) \in R_p^{2\ell}$, and $\mathsf{pp}$.
**Witness**: $\boldsymbol{s}, \boldsymbol{e}_{\mathsf{ek}}, \boldsymbol{f}_{\mathsf{rlk}} \in R_p$, $\vec{\boldsymbol{e}}_{\mathsf{rlk}} \in R_p^{3\ell}$, and $\vec{\boldsymbol{e}}_{\mathsf{atk}} \in R_p^{2\ell}$.

1. The prover $\mathtt{P}$ samples the following.

$$\hat{\boldsymbol{s}} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{s})),\ \underline{\hat{\boldsymbol{s}}} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{s}))$$
$$\underline{\hat{\boldsymbol{s}}}_{\mathsf{atk},0} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{s}(X^{-1}))),\ \underline{\hat{\boldsymbol{s}}}_{\mathsf{atk},1} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{s}(X^5)))$$
$$\hat{\boldsymbol{f}}_{\mathsf{rlk}} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{f}_{\mathsf{rlk}})),\ \underline{\hat{\boldsymbol{f}}}_{\mathsf{rlk}} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{f}_{\mathsf{rlk}}))$$
$$\hat{\boldsymbol{e}}_{\mathsf{ek}} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{e}_{\mathsf{ek}})),\ \underline{\hat{\boldsymbol{e}}}_{\mathsf{ek}} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{e}_{\mathsf{ek}}))$$
$$\hat{\boldsymbol{e}}_{\mathsf{rlk},i,j} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{e}_{\mathsf{rlk},i,j})),\ \underline{\hat{\boldsymbol{e}}}_{\mathsf{rlk},i,j} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{e}_{\mathsf{rlk},i,j}))$$
$$\hat{\boldsymbol{e}}_{\mathsf{atk},i,j} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{e}_{\mathsf{atk},i,j})),\ \underline{\hat{\boldsymbol{e}}}_{\mathsf{atk},i,j} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{e}_{\mathsf{atk},i,j}))$$

Then, the prover sends polynomial oracles $[\![\hat{\boldsymbol{s}}]\!]$, $[\![\underline{\hat{\boldsymbol{s}}}]\!]$, $[\![\underline{\hat{\boldsymbol{s}}}_{\mathsf{atk},0}]\!]$, $[\![\underline{\hat{\boldsymbol{s}}}_{\mathsf{atk},1}]\!]$, $[\![\hat{\boldsymbol{f}}_{\mathsf{rlk}}]\!]$, $[\![\underline{\hat{\boldsymbol{f}}}_{\mathsf{rlk}}]\!]$, $[\![\hat{\boldsymbol{e}}_{\mathsf{ek}}]\!]$, $[\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{ek}}]\!]$, $\{[\![\hat{\boldsymbol{e}}_{\mathsf{rlk},i,j}]\!],\ [\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{rlk},i,j}]\!]\}_{0\leq i<3, 0\leq j<\ell}$, $\{[\![\hat{\boldsymbol{e}}_{\mathsf{atk},i,j}]\!],\ [\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{atk},i,j}]\!]\}_{0\leq i<2, 0\leq j<\ell}$ of degree $< L$ to the verifier $\mathtt{V}$.

2. The verifier $\mathtt{V}$ sends a random challenge $\delta \leftarrow \mathcal{U}(\mathbb{Z}_p \setminus \mathbb{H})$.
3. For the smallness of witness, the prover $\mathtt{P}$ and the verifier $\mathtt{V}$ invoke the following PIOP, where $B$ is a norm bound of $\chi$ and $\psi$.

$$\Pi_{\mathtt{NC}}\Big(B; [\![\hat{\boldsymbol{s}}]\!], [\![\hat{\boldsymbol{f}}_{\mathsf{rlk}}]\!], [\![\hat{\boldsymbol{e}}_{\mathsf{ek}}]\!], \{[\![\hat{\boldsymbol{e}}_{\mathsf{rlk},i,j}]\!]\}, \{[\![\hat{\boldsymbol{e}}_{\mathsf{atk},i,j}]\!]\}\Big)$$

4. For the validity of NTT representations, the prover $\mathtt{P}$ and the verifier $\mathtt{V}$ invoke the following PIOP, where $T$ is the Vandermonde matrix for $\mathtt{NTT}$.

$$\begin{aligned}\Pi_{\mathtt{LC}}\Big(T; &[\![\hat{\boldsymbol{s}}]\!], [\![\hat{\boldsymbol{f}}_{\mathsf{rlk}}]\!], [\![\hat{\boldsymbol{e}}_{\mathsf{ek}}]\!], \{[\![\hat{\boldsymbol{e}}_{\mathsf{rlk},i,j}]\!]\}, \{[\![\hat{\boldsymbol{e}}_{\mathsf{atk},i,j}]\!]\},\\ &[\![\underline{\hat{\boldsymbol{s}}}]\!], [\![\underline{\hat{\boldsymbol{f}}}_{\mathsf{rlk}}]\!], [\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{ek}}]\!], \{[\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{rlk},i,j}]\!]\}, \{[\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{atk},i,j}]\!]\}\Big)\end{aligned}$$

5. For the validity of the encryption key share, the prover $\mathtt{P}$ and the verifier $\mathtt{V}$ invoke the following PIOPs for $\underline{\vec{p}} = \mathtt{NTT}(\boldsymbol{p})$ and $\underline{\vec{u}}_{\mathsf{ek}} = \mathtt{NTT}(\boldsymbol{u}_{\mathsf{ek}})$.

$$\Pi_{\mathtt{RC}}\left(\underline{\vec{p}} + \underline{\vec{u}}_{\mathsf{ek}} \circ \vec{X}_0 - \vec{X}_1; [\![\underline{\hat{\boldsymbol{s}}}]\!], [\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{ek}}]\!]\right)$$

6. For the validity of the relinearization key share, the prover $\mathtt{P}$ and the verifier $\mathtt{V}$ invoke the following PIOP, where $\underline{\vec{r}}_{i,j} = \mathtt{NTT}(\boldsymbol{r}_{i,j})$, $\underline{\vec{u}}_{\mathsf{rlk},i,j} = \mathtt{NTT}(\boldsymbol{u}_{\mathsf{rlk},i,j})$ for $0 \le i < 3$, $0 \le j < \ell$.

$$\begin{aligned}\Pi_{\mathtt{RC}}\Bigg(&\sum_{j=0}^{\ell-1} \delta^j \cdot (\underline{\vec{r}}_{0,j} + \underline{\vec{u}}_{\mathsf{rlk},0,j} \circ \vec{X}_0 - \vec{X}_{j+2})\\ &+ \sum_{j=0}^{\ell-1} \delta^{j+\ell} \cdot (\underline{\vec{r}}_{1,j} + \underline{\vec{u}}_{\mathsf{rlk},1,j} \circ \vec{X}_1 - \lfloor p/q_j \rceil \cdot \vec{X}_0 - \vec{X}_{j+\ell+2})\\ &+ \sum_{j=0}^{\ell-1} \delta^{j+2\ell} \cdot (\underline{\vec{r}}_{2,j} + \underline{\vec{u}}_{\mathsf{rlk},2,j} \circ \vec{X}_0 + \lfloor p/q_j \rceil \cdot \vec{X}_1 - \vec{X}_{j+2\ell+2})\\ &; [\![\underline{\hat{\boldsymbol{s}}}]\!], [\![\underline{\hat{\boldsymbol{f}}}_{\mathsf{rlk}}]\!], \{[\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{rlk},i,j}]\!]\}\Bigg)\end{aligned}$$

7. For the validity of the automorphism key, the prover $\mathtt{P}$ and the verifier $\mathtt{V}$ invoke the following PIOPs, where $\underline{\vec{a}}_{i,j} = \mathtt{NTT}(\boldsymbol{a}_{i,j})$, $\underline{\vec{u}}_{\mathsf{atk},i,j} = \mathtt{NTT}(\boldsymbol{u}_{\mathsf{atk},i,j})$ for $0 \le i < 2$, $0 \le j < \ell$, and $P_0$ and $P_1$ are permutation matrices for the automorphisms $X \mapsto X^{-1}$ and $X \mapsto X^5$, respectively.

$$\begin{aligned}&\Pi_{\mathtt{LC}}\Big(P_0; [\![\underline{\hat{\boldsymbol{s}}}]\!], [\![\underline{\hat{\boldsymbol{s}}}_{\mathsf{atk},0}]\!]\Big),\ \Pi_{\mathtt{LC}}\Big(P_1; [\![\underline{\hat{\boldsymbol{s}}}]\!], [\![\underline{\hat{\boldsymbol{s}}}_{\mathsf{atk},1}]\!]\Big),\\ &\Pi_{\mathtt{RC}}\Bigg(\sum_{j=0}^{\ell-1} \delta^j \cdot (\underline{\vec{a}}_{0,j} + \underline{\vec{u}}_{\mathsf{atk},0,j} \circ \vec{X}_0 - \lfloor p/q_j \rceil \cdot \vec{X}_1 - \vec{X}_{j+3})\\ &\quad + \sum_{j=0}^{\ell-1} \delta^{j+\ell} \cdot (\underline{\vec{a}}_{1,j} + \underline{\vec{u}}_{\mathsf{atk},1,j} \circ \vec{X}_0 - \lfloor p/q_j \rceil \cdot \vec{X}_2 - \vec{X}_{j+\ell+3})\\ &\quad ; [\![\underline{\hat{\boldsymbol{s}}}]\!], [\![\underline{\hat{\boldsymbol{s}}}_{\mathsf{atk},0}]\!], [\![\underline{\hat{\boldsymbol{s}}}_{\mathsf{atk},1}]\!], \{[\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{atk},i,j}]\!]\}\Bigg)\end{aligned}$$

Fig. 4: PIOP for Setup Phase

### 4.4 PIOP for Encryption Phase

In the encryption phase of Fig. 1, we need to prove the validity of the input ciphertexts shared during this phase to ensure the correctness of the subsequent evaluation phase. For efficient proof generation, we first generate ciphertexts in $R_p$ and then apply modulus switching to $R_q$ for performing homomorphic evaluation during the evaluation phase. To achieve this, we modify the encryption algorithm as follows.

– $\underline{\mathtt{Enc}}$: Given a joint encryption key $\mathsf{ek} = \boldsymbol{p} \in R_p$ and a plaintext $\boldsymbol{m} \in R_t$, a ciphertext $\mathsf{ct} \in R_p^2$ is generated as follows.

$$\mathsf{ct} = \boldsymbol{f} \cdot (\boldsymbol{p}, \boldsymbol{u}_{\mathsf{ek}}) + (\lfloor p/t \rceil \cdot \boldsymbol{m} + \boldsymbol{e}_0, \boldsymbol{e}_1) \pmod{p} \tag{7}$$

Next, we need to prove that $\mathsf{ct} = (\boldsymbol{c}_0, \boldsymbol{c}_1) \in R_p^2$ remains valid after modulus switching to $R_q$. Specifically, we need to show that for some small-normed $\boldsymbol{e}'$, the following equation holds, where $\boldsymbol{s}^{(i)} = \mathsf{dsk}^{(i)}$.

$$\left\lfloor \frac{q}{p}\boldsymbol{c}_0 \right\rceil + \left\lfloor \frac{q}{p}\boldsymbol{c}_1 \right\rceil \cdot \sum_{i \in I} \boldsymbol{s}^{(i)} = \lfloor q/t \rceil \cdot \boldsymbol{m} + \boldsymbol{e}' \pmod{q}$$

Assuming the joint encryption key $\mathsf{ek}$ is verified through $\Pi_{\mathtt{DKG}}$ and $\mathsf{ct}$ satisfies Eq. (7), we obtain the following result, which ensures the validity of $\mathsf{ct}$ after modulus switching.

**Lemma 3.** *For* $\left\lfloor \frac{q}{p}\mathsf{ct} \right\rceil \in R_q^2$, *it holds that* $\|\boldsymbol{e}'\|_\infty \leq \frac{q}{p}(2NB^2+B)|I| + \frac{1}{2}\left(\frac{q}{p}2t + NB|I| + 1\right)$ *if* $\mathsf{ek}$ *is verified through* $\Pi_{\mathtt{DKG}}$, *and* $\|\boldsymbol{e}_0\|_\infty$, $\|\boldsymbol{e}_1\|_\infty$, $\|\boldsymbol{f}\|_\infty \leq B$.

Therefore, by verifying the satisfiability of Eq. (7) and the smallness of $\boldsymbol{e}_0$, $\boldsymbol{e}_1$, $\boldsymbol{f}$, we can guarantee the validity of $\left\lfloor \frac{q}{p}\mathsf{ct} \right\rceil \in R_q^2$. Below, we provide a complete description of the PIOP for the validity of encryption, along with its complexity analysis.

**Theorem 5 (PIOP for Encryption Phase).** $\Pi_{\mathtt{Enc}}$ *is an HVZK PIOP with the following complexity, where* $k_1 = \lfloor \log B \rfloor + 1$ *and* $k_2 = \lfloor \log t \rfloor + 1$.

– *The soundness error is* $O(\frac{k_1+k_2+N}{p-N})$
– *The prover time is* $O((k_1+k_2)N \log N)$
– *The verifier time is* $O((k_1+k_2)N \log N)$
– *The query complexity is* $3k_1 + k_2 + 30$ *and the total number of distinct query points is* 1.
– *The size of proof oracles is* $\leq (3k_1 + k_2 + 17)L + 2N$ *elements in* $\mathbb{Z}_p$.
– *The size of witness is* $4N$ *elements in* $\mathbb{Z}_p$.

$\Pi_{\mathtt{Enc}}$

**Instance**: $\mathsf{ek} = \boldsymbol{p} \in R_p$, $\mathsf{ct} = (\boldsymbol{c}_0, \boldsymbol{c}_1) \in R_p^2$, and $\mathsf{pp}$.
**Witness**: $\boldsymbol{e}_0, \boldsymbol{e}_1, \boldsymbol{f}, \boldsymbol{m} \in R_p$.

1. The prover $\mathtt{P}$ samples the following.

$$\hat{\boldsymbol{e}}_0 \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{e}_0)),\ \underline{\hat{\boldsymbol{e}}}_0 \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{e}_0))$$
$$\hat{\boldsymbol{e}}_1 \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{e}_1)),\ \underline{\hat{\boldsymbol{e}}}_1 \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{e}_1))$$
$$\hat{\boldsymbol{f}} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{f})),\ \underline{\hat{\boldsymbol{f}}} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{f}))$$
$$\hat{\boldsymbol{m}} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{m})),\ \underline{\hat{\boldsymbol{m}}} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{m}))$$

   Then, the prover $\mathtt{P}$ sends polynomial oracles $[\![\hat{\boldsymbol{e}}_0]\!]$, $[\![\underline{\hat{\boldsymbol{e}}}_0]\!]$, $[\![\hat{\boldsymbol{e}}_1]\!]$, $[\![\underline{\hat{\boldsymbol{e}}}_1]\!]$, $[\![\hat{\boldsymbol{f}}]\!]$, $[\![\underline{\hat{\boldsymbol{f}}}]\!]$, $[\![\hat{\boldsymbol{m}}]\!]$, $[\![\underline{\hat{\boldsymbol{m}}}]\!]$ of degree $\leq L$ to the verifier $\mathtt{V}$.
2. The verifier $\mathtt{V}$ sends a random challenge $\delta \leftarrow \mathcal{U}(\mathbb{Z}_p \setminus \mathbb{H})$.
3. For the smallness of witenss, the prover $\mathtt{P}$ and the verifier $\mathtt{V}$ invoke the following PIOPs, where $B$ is a norm bound of $\chi$ and $\psi$.

$$\Pi_{\mathtt{NC}}\Big(B; [\![\hat{\boldsymbol{e}}_0]\!], [\![\hat{\boldsymbol{e}}_1]\!], [\![\hat{\boldsymbol{f}}]\!]\Big),\ \Pi_{\mathtt{NC}}\Big(t; [\![\hat{\boldsymbol{m}}]\!]\Big)$$

4. For the validity of NTT representations, the prover $\mathtt{P}$ and the verifier $\mathtt{V}$ invoke the following PIOP, where $T$ is the Vandermonde matrix for $\mathtt{NTT}$.

$$\Pi_{\mathtt{LC}}\Big(T; [\![\hat{\boldsymbol{e}}_0]\!], [\![\hat{\boldsymbol{e}}_1]\!], [\![\hat{\boldsymbol{f}}]\!], [\![\hat{\boldsymbol{m}}]\!], [\![\underline{\hat{\boldsymbol{e}}}_0]\!], [\![\underline{\hat{\boldsymbol{e}}}_1]\!], [\![\underline{\hat{\boldsymbol{f}}}]\!], [\![\underline{\hat{\boldsymbol{m}}}]\!]\Big)$$

5. For the validity of the ciphertext $\mathsf{ct}$, the prover $\mathtt{P}$ and the verifier $\mathtt{V}$ invoke the following PIOP, where $\underline{\vec{p}} = \mathtt{NTT}(\boldsymbol{p})$, $\underline{\vec{u}}_{\mathsf{ek}} = \mathtt{NTT}(\boldsymbol{u}_{\mathsf{ek}})$, $\underline{\vec{c}}_0 = \mathtt{NTT}(\boldsymbol{c}_0)$, and $\underline{\vec{c}}_1 = \mathtt{NTT}(\boldsymbol{c}_1)$.

$$\Pi_{\mathtt{RC}}\Big((\underline{\vec{c}}_0 - \underline{\vec{p}} \circ \vec{X}_0 - \lfloor p/t \rceil \cdot \vec{X}_1 - \vec{X}_2) + \delta \cdot (\underline{\vec{c}}_1 - \underline{\vec{u}}_{\mathsf{ek}} \circ \vec{X}_0 - \vec{X}_3)$$
$$; [\![\underline{\hat{\boldsymbol{f}}}]\!], [\![\underline{\hat{\boldsymbol{m}}}]\!], [\![\underline{\hat{\boldsymbol{e}}}_0]\!], [\![\underline{\hat{\boldsymbol{e}}}_1]\!]\Big)$$

Fig. 5: PIOP for Encryption Phase

### 4.5 PIOP for Decryption Phase

In the decryption phase of Fig. 1, it is necessary to validate the partial decryptions from each party to ensure the correctness of the joint decryption. This process includes verifying the well-formedness of each partial decryption and checking the compliance of the secret key share with the encryption key share. To enable efficient proof generation, we modify the setup and decryption algorithms as follows.

- $\underline{\mathtt{Setup}}$: Generate a common random string $\boldsymbol{u}_{\mathtt{DD}} \leftarrow \mathcal{U}(R_{B_{\mathtt{DD}}})$.
- $\underline{\mathtt{DistDec}}$: Given a ciphertext $\mathsf{ct} = (\boldsymbol{c}_0, \boldsymbol{c}_1) \in R_p^2$, and a secret key share $\mathsf{dsk} = \boldsymbol{s}$, sample $\boldsymbol{e}_{\mathtt{DD}} \leftarrow \psi$, $\boldsymbol{f}_{\mathtt{DD}} \leftarrow \chi$, and compute $\boldsymbol{d} = \boldsymbol{c}_1\boldsymbol{s} + [\boldsymbol{u}_{\mathtt{DD}}\boldsymbol{f}_{\mathtt{DD}} + \boldsymbol{e}_{\mathtt{DD}}]_{B_{\mathtt{DD}}} \pmod p$, then return the partial decryption $\mathsf{dct} = \boldsymbol{d}$.

– $\underline{\texttt{JointDec}}$: Return the output plaintext $\boldsymbol{m} = \left\lfloor \frac{t}{p} \left( \boldsymbol{c}_0 + \sum_{i \in I} \mathsf{dct}^{(i)} \right) \right\rceil \pmod t$.

In the above modifications, we not only change the ciphertext space from $R_q$ to $R_p$, but also replace random sampling $\boldsymbol{e} \leftarrow \mathcal{U}(R_{\mathtt{DD}})$ with $[\boldsymbol{u}_{\mathtt{DD}} \boldsymbol{f}_{\mathtt{DD}} + \boldsymbol{e}_{\mathtt{DD}}]_{B_{\mathtt{DD}}}$. This change is made to bypass the overhead of proving the norm bound of $\boldsymbol{e}$. To be precise, as analyzed in Theorem 3, the complexity of norm check PIOPs scales linearly with the logarithmic size of the norm bound. However, the bound $B_{\mathtt{DD}}$ of the noise $\boldsymbol{e}$ can be hundreds of bits. Thus, directly proving the bound of $\boldsymbol{e}$ would result in significant performance degradation.

To address this issue, we utilize the optimization technique used in [CMS+23], where $\boldsymbol{e}$ is sampled as an RLWE sample over $R_{B_{\mathtt{DD}}}$, rather than directly from $\mathcal{U}(R_{B_{\mathtt{DD}}})$. To be precise, we sample the noise $\boldsymbol{e}$ by $\boldsymbol{u}_{\mathtt{DD}} \boldsymbol{f}_{\mathtt{DD}} + \boldsymbol{e}_{\mathtt{DD}} \pmod{B_{\mathtt{DD}}}$, where $\boldsymbol{u}_{\mathtt{DD}}$ is a CRS sampled from $\mathcal{U}(R_{B_{\mathtt{DD}}})$, $\boldsymbol{f}_{\mathtt{DD}} \leftarrow \chi$, and $\boldsymbol{e}_{\mathtt{DD}} \leftarrow \psi$. However, as we work in the ciphertext space $R_p$, there exists $\boldsymbol{k} \in R_p$ such that $\|\boldsymbol{k}\|_\infty \leq B(N+1)$, and the following holds.

$$B_{\mathtt{DD}} \cdot \boldsymbol{k} = \boldsymbol{u}_{\mathtt{DD}} \boldsymbol{f}_{\mathtt{DD}} + \boldsymbol{e}_{\mathtt{DD}} - [\boldsymbol{u}_{\mathtt{DD}} \boldsymbol{f}_{\mathtt{DD}} + \boldsymbol{e}_{\mathtt{DD}}]_{B_{\mathtt{DD}}} \pmod p \tag{8}$$

Then, proving the bound of $\boldsymbol{e}$ can be replaced by proving the satisfiability of Eq. (8) and the bounds of $\boldsymbol{e}_{\mathtt{DD}}$, $\boldsymbol{f}_{\mathtt{DD}}$, and $\boldsymbol{k}$, whose complexity is independent of the size of the norm bound $B_{\mathtt{DD}}$. As a result, we can effectively reduce the cost of proving the norm bound of $\boldsymbol{e}$, especially when $B_{\mathtt{DD}}$ is large.

Below, we provide a complete description of the PIOP for the decryption phase, along with a complexity analysis.

**Theorem 6 (PIOP for Decryption Phase).** *$\Pi_{\mathtt{DD}}$ is an HVZK PIOP with the following complexity, where $k_1 = \lfloor \log B \rfloor + 1$ and $k_3 = \lfloor \log((N+1)B) \rfloor + 1$.*

- *The soundness error is $O\left(\frac{k_1 + k_3 + N}{p - N}\right)$.*
- *The prover time is $O((k_1 + k_3) N \log N)$.*
- *The verifier time is $O(N \log N + k_1 + k_3)$.*
- *The query complexity is $4k_1 + k_3 + 36$.*
- *The size of proof oracles is $\leq (4k_1 + k_3 + 19)L + 2N$ elements in $\mathbb{Z}_p$.*
- *The size of witness is $4N$ elements in $\mathbb{Z}_p$.*

$\Pi_{\mathtt{DD}}$

**Instance**: $\mathsf{ek} = \boldsymbol{p} \in R_p$, $\mathsf{ct} = (\boldsymbol{c}_0, \boldsymbol{c}_1) \in R_p^2$, $\mathsf{dct} = \boldsymbol{d} \in R_p$, and $\mathsf{pp}$.
**Witness**: $\mathsf{dsk} = \boldsymbol{s}, \boldsymbol{e}_{\mathsf{ek}}, \boldsymbol{e}_{\mathtt{DD}}, \boldsymbol{f}_{\mathtt{DD}} \in R_p$

1. Given the input witness polynomials, the prover $\mathtt{P}$ computes $\boldsymbol{k} \in R_p$, which satisfies,
$$B_{\mathtt{DD}} \cdot \boldsymbol{k} = \boldsymbol{u}_{\mathtt{DD}} \boldsymbol{f}_{\mathtt{DD}} + \boldsymbol{e}_{\mathtt{DD}} - [\boldsymbol{u}_{\mathtt{DD}} \boldsymbol{f}_{\mathtt{DD}} + \boldsymbol{e}_{\mathtt{DD}}]_{B_{\mathtt{DD}}} \pmod p$$

and sample the following.

$$\hat{\boldsymbol{s}} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{s})),\ \underline{\hat{\boldsymbol{s}}} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{s}))$$
$$\hat{\boldsymbol{e}}_{\mathsf{ek}} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{e}_{\mathsf{ek}})),\ \underline{\hat{\boldsymbol{e}}}_{\mathsf{ek}} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{e}_{\mathsf{ek}}))$$
$$\hat{\boldsymbol{e}}_{\mathtt{DD}} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{e}_{\mathtt{DD}})),\ \underline{\hat{\boldsymbol{e}}}_{\mathtt{DD}} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{e}_{\mathtt{DD}}))$$
$$\hat{\boldsymbol{f}}_{\mathtt{DD}} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{f}_{\mathtt{DD}})),\ \underline{\hat{\boldsymbol{f}}}_{\mathtt{DD}} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{f}_{\mathtt{DD}}))$$
$$\hat{\boldsymbol{k}} \leftarrow \mathtt{REcd}(\mathtt{Coeff}(\boldsymbol{k})),\ \underline{\hat{\boldsymbol{k}}} \leftarrow \mathtt{REcd}(\mathtt{NTT}(\boldsymbol{k}))$$

Then, the prover P sends polynomial oracles $[\![\hat{\boldsymbol{s}}]\!]$, $[\![\underline{\hat{\boldsymbol{s}}}]\!]$, $[\![\hat{\boldsymbol{e}}_{\mathsf{ek}}]\!]$, $[\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{ek}}]\!]$, $[\![\hat{\boldsymbol{e}}_{\mathtt{DD}}]\!]$, $[\![\underline{\hat{\boldsymbol{e}}}_{\mathtt{DD}}]\!]$, $[\![\hat{\boldsymbol{f}}_{\mathtt{DD}}]\!]$, $[\![\underline{\hat{\boldsymbol{f}}}_{\mathtt{DD}}]\!]$, $[\![\hat{\boldsymbol{k}}]\!]$, $[\![\underline{\hat{\boldsymbol{k}}}]\!]$ of degree $\leq L$ to the verifier V.

2. The verifier V sends a random challenge $\delta \leftarrow \mathcal{U}(\mathbb{Z}_p \setminus \mathbb{H})$.
3. For the smallness of witness, the prover P and the verifier V invoke the following PIOPs, where $B$ is a norm bound of $\chi$ and $\psi$.

$$\Pi_{\mathtt{NC}}\Big(B; [\![\hat{\boldsymbol{s}}]\!], [\![\hat{\boldsymbol{e}}_{\mathsf{ek}}]\!], [\![\hat{\boldsymbol{e}}_{\mathtt{DD}}]\!], [\![\hat{\boldsymbol{f}}_{\mathtt{DD}}]\!]\Big),\ \Pi_{\mathtt{NC}}\Big(B(N+1); [\![\hat{\boldsymbol{k}}]\!]\Big)$$

4. For the validity of NTT representations, the prover P and the verifier V invoke the following PIOP, where $T$ is the Vandermonde matrix for NTT.

$$\Pi_{\mathtt{LC}}\Big(T; [\![\hat{\boldsymbol{s}}]\!], [\![\hat{\boldsymbol{e}}_{\mathsf{ek}}]\!], [\![\hat{\boldsymbol{e}}_{\mathtt{DD}}]\!], [\![\hat{\boldsymbol{f}}_{\mathtt{DD}}]\!], [\![\hat{\boldsymbol{k}}]\!], [\![\underline{\hat{\boldsymbol{s}}}]\!], [\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{ek}}]\!], [\![\underline{\hat{\boldsymbol{e}}}_{\mathtt{DD}}]\!], [\![\underline{\hat{\boldsymbol{f}}}_{\mathtt{DD}}]\!], [\![\underline{\hat{\boldsymbol{k}}}]\!],\Big)$$

5. For the validity of the partial decryption dct, the prover P and the verifier V invoke the following PIOP, where $\vec{\underline{p}} = \mathtt{NTT}(\boldsymbol{p})$, $\vec{\underline{u}}_{\mathsf{ek}} = \mathtt{NTT}(\boldsymbol{u}_{\mathsf{ek}})$, $\vec{\underline{u}}_{\mathtt{DD}} = \mathtt{NTT}(\boldsymbol{u}_{\mathtt{DD}})$, $\vec{\underline{c}}_0 = \mathtt{NTT}(\boldsymbol{c}_0)$, $\vec{\underline{c}}_1 = \mathtt{NTT}(\boldsymbol{c}_1)$, and $\vec{\underline{d}} = \mathtt{NTT}(\boldsymbol{d})$.

$$\Pi_{\mathtt{RC}}\Big(\vec{\underline{p}} + \vec{\underline{u}}_{\mathsf{ek}} \circ \vec{X}_0 - \vec{X}_1 + \delta \cdot (\vec{\underline{d}} - \vec{\underline{c}}_1 \circ \vec{X}_0 - \vec{\underline{u}}_{\mathtt{DD}} \circ \vec{X}_2 - \vec{X}_3 + B_{\mathtt{DD}} \cdot \vec{X}_4)$$
$$; [\![\underline{\hat{\boldsymbol{s}}}]\!], [\![\underline{\hat{\boldsymbol{e}}}_{\mathsf{ek}}]\!], [\![\underline{\hat{\boldsymbol{f}}}_{\mathtt{DD}}]\!], [\![\underline{\hat{\boldsymbol{e}}}_{\mathtt{DD}}]\!], [\![\underline{\hat{\boldsymbol{k}}}]\!]\Big)$$

Fig. 6: PIOP for Decryption Phase

## 5 Evaluation

In this section, we compile the PIOPs for MPHE from Section 4 into ZKAoKs using a polynomial commitment scheme (PCS). We begin by describing how we compile our PIOPs, including the selection of the PCS and the parameter settings for both MPHE and the PCS. We then present experimental results for the compiled ZKAoKs, providing concrete proof sizes and runtimes for both the prover and the verifier.

### 5.1 Implementation Details

**Polynomial Commitment.** To compile the PIOPs for MPHE, we require a PCS that supports the hiding property and large NTT-friendly prime fields, i.e., $2N \mid p - 1$ and $p \approx q$. Among the possible candidates, we use the lattice-based

PCS by Hwang et al. [HSS24], referred to as the HSS scheme, since it offers fast proving performance and supports large prime fields. Additionally, it provides a transparent setup and is plausibly post-quantum secure. We note that other PCS schemes satisfying these conditions can be used, due to the modularity of PIOPs. In the HSS scheme, the prime field $\mathbb{Z}_p$ is supported, where the modulus $p$ takes the form $b^r + 1$, with $r$ being a power of two. Thus, by choosing an appropriate even number $b$ such that $p = b^r + 1$ becomes a prime and $p \approx q$, it automatically becomes NTT-friendly since $2^r \mid p - 1$.

**MPHE**. For the parameters of MPHE, we need to choose the key distribution $\chi$, the noise distribution $\psi$, the ciphertext modulus $q$, and the ring degree $N$. For both $\chi$ and $\psi$, we use the uniform ternary distribution $\mathcal{U}(R_3)$ to attain the minimum bound $B = 1$. This choice of distributions is commonly used in zero-knowledge proof systems for RLWE samples [ALS20, ENS20, BLS19] to reduce the cost of proof generation. However, we note that our PIOP construction can be easily generalized to larger values of $B$, with the overhead growing as $\log B$, as analyzed in Theorem 3.

For the ring dimension $N$, we use $2^{14}$ and $2^{15}$, which provide sufficiently large ciphertext moduli $q$ to enable general homomorphic evaluations, including BFV bootstrapping [KSS24, LW24, GIKV23]. For the plaintext modulus $t$, we choose $2^{16} + 1$ for both cases, as it is the most widely used plaintext modulus for BFV due to the SIMD structure $\mathbb{Z}_t^N \cong R_t$ for the plaintext space. For the ciphertext modulus $q$, we choose the largest value for each ring dimension, ensuring 128-bit security as estimated by the lattice estimator [APS15]. For the RNS primes $q_i$, we use 50–60 bit-sized primes to minimize the length $\ell$ of the moduli chain. Specifically, we select 53–54 bit primes to achieve levels $\ell = 8$ and $\ell = 16$ for ring dimensions $N = 2^{14}$ and $N = 2^{15}$, respectively. For the temporal modulus $p = b^r + 1$, we fix $r = 32$ for $N = 2^{14}$ and $r = 64$ for $N = 2^{15}$, and find the appropriate value of $b$ such that $p$ is prime and $p \approx q$. In Table 1, we summarize the parameter settings.

| | $N$ | $t$ | $\lceil \log q \rceil$ | $\lceil \log p \rceil$ | $\ell$ | $b$ | $r$ |
|---|---|---|---|---|---|---|---|
| Params I | $2^{14}$ | $2^{16}+1$ | 429 | 429 | 8 | 10792 | 32 |
| Params II | $2^{15}$ | $2^{16}+1$ | 865 | 865 | 16 | 11710 | 64 |

Table 1: Parameter Sets

**Complexity.** As described in Theorem 7, when compiling PIOP into ZKAoKs, additional overhead from the PCS is added to the complexity of both the prover and verifier, as well as to the proof size. In the HSS scheme, the prover complexity is quasi-linear in the witness size, while the verifier complexity and the proof size are square-root in the witness size. In Theorems 4 to 6, the prover and verifier complexity is already quasi-linear in the witness size, so the asymptotic complexity remains unchanged after compilation. For the proof size, since the

number of query points is constant, the proof size becomes square-root in the witness size.

### 5.2 Benchmark Results

To demonstrate the concrete efficiency of our PIOP for MPHE, we implement our PIOPs at a proof-of-concept level through PIOP compilation via the HSS polynomial commitment scheme. The parameters for the HSS scheme are chosen to provide the smallest proof size while ensuring 128-bit security. The detailed parameter sets are deferred to Appendix D. All experiments were performed with a single thread on a machine with an Intel(R) Xeon(R) Platinum 8268 CPU running at 2.90GHz and 384GB of RAM, unless otherwise specified. Our source code is available at `https://github.com/SNUCP/buckler`.

**Malicious Compiler.** First, we measure the performance of $\Pi_{\texttt{DKG}}$, $\Pi_{\texttt{Enc}}$, and $\Pi_{\texttt{DD}}$, which are required to compile passively secure MPHE-based MPC protocols into maliciously secure ones. We note that the malicious compiler is general, so any MPHE-based MPC protocol can be compiled into a maliciously secure one at the cost of generating ZKAoKs for each phase. In Table 2, we evaluate the proof generation speed, verification speed, and proof size, along with the size of shares originally transmitted during each phase in the passively secure protocol.

| | Phase | Prove (s) | Verify (s) | Proof Size (MB) | Share Size (MB) |
|---|---|---|---|---|---|
| Params I | Setup | 139 | 12.3 | 34.7 | 34.4 |
| | Enc | 41.1 | 2.74 | 15.8 | 1.7 |
| | Dec | 34.9 | 2.61 | 16.4 | 0.8 |
| Params II | Setup | 921 | 61.3 | 86 | 273.7 |
| | Enc | 156 | 6.26 | 29.2 | 6.8 |
| | Dec | 136 | 6.36 | 30.7 | 3.4 |

Table 2: Benchmark results for the Malicious Compiler

For the setup phase, the shares are public key shares, which consist of $(5\ell+1)$ elements in $R_p$, and the benchmark results show that the proof size remains similar to the share size when $N = 2^{14}$ but becomes about 2.7 times smaller than the share size when $N = 2^{15}$. We attribute this to the square root complexity of the HSS scheme in proof size. We expect the gap between the share size and proof size to grow further for larger parameters, such as $N = 2^{16}$.

For the encryption phase, the shares are input ciphertexts, each consisting of two elements in $R_p$. For the decryption phase, the shares are partial decryptions, each consisting of one element in $R_p$. In the benchmark results, our implementation yields larger proof sizes compared to the share sizes. This is due to the overhead from the norm check protocol for the bounds $t$ and $B(N+1)$, respectively,

as their logarithmic size affects the complexity, as described in Theorems 5 and 6. However, since our proof size has square root complexity, it eventually becomes smaller as the size of input and output grows. For example, if the MPHE-based MPC protocol in Fig. 1 is modified to send 64 ciphertexts for the encryption phase, our implementation yields a proof size of 303 MB for encryption when $N = 2^{15}$, which is about 8 times larger compared to the single ciphertext case. However, the ciphertext size grows to 435 MB, which is about 1.4 times larger than the proof size. Thus, we expect the overhead from proof size to become smaller as the protocol handles more input and output ciphertexts.

Finally, our implementation supports efficient parallelism when utilizing multiple threads. As described in Table 3, the timing of proof generation achieve speedup of 3.3x and 11.7x when utilizing 4 and 16 threads, respectively. We attribute this improvement to the high parallelism of lattice-based cryptography. Thus, computational overheads for proof generation and verification can be effectively mitigated with the aid of parallelism.

| | Phase | Prove (4 threads) | | Prove (16 threads) | |
|---|---|---|---|---|---|
| | | Time (s) | Speedup | Time (s) | Speedup |
| Params I | Setup | 45.4 | 3.1x | 14.0 | 9.9x |
| | Enc | 13.1 | 3.1x | 5.33 | 7.7x |
| | Dec | 11.3 | 3.1x | 4.82 | 7.2x |
| Params II | Setup | 279 | 3.3x | 78.5 | 11.7x |
| | Enc | 48.1 | 3.2x | 17.3 | 9.0x |
| | Dec | 41.4 | 3.3x | 15.9 | 8.6x |

Table 3: Benchmark Results for Speedup from Parallelism

**Comparison with PELTA.** To demonstrate the efficiency of our implementation over prior works, we provide benchmark results dedicated to a comparison with PELTA [CMS+23], the state-of-the-art ZKAoKs for MPHE. Since only the benchmark results for the encryption key are available in [CMS+23] for the setup phase, we modify $\Pi_{\mathtt{DKG}}$ to validate encryption key shares only. This modification results in proof oracles of size $\leq (2k+10)L+2N$ elements in $\mathbb{Z}_p$. For the benchmark results of PELTA, we borrow timing results and proof sizes from Table 4 in [CMS+23] and multiply them by $\ell$ since the performance results are measured for a single subring $R_{q_i}$ and grow linearly with respect to $\ell$, as shown in Table 5 in [CMS+23].

As shown in Table 4, our implementation achieves a speedup of 53.8x and 264.2x for proof generation and verification when $N = 2^{14}$, and a speedup of 111.3x and 768.5x for proof generation and verification when $N = 2^{15}$, respectively. We attribute this to our optimization technique via modulus switching,

which effectively reduces the overhead from handling each $R_{q_i}$. For proof size, our implementation achieves a 2x and 3x improvement for $N = 2^{14}$ and $N = 2^{15}$, respectively. This improvement is due to the square root complexity of proof size in our implementation. Therefore, compared to the previous state-of-the-art solution, our approach achieves significant improvements across all aspects.

| | $N$ | $\ell$ | Prove (s) | Verify (s) | Proof Size (MB) |
|---|---|---|---|---|---|
| PELTA | $2^{14}$ | 8 | 732 | 370 | 26.4 |
| | $2^{15}$ | 16 | 5710 | 2613 | 92.8 |
| Ours | $2^{14}$ | 8 | 13.6 | 1.40 | 9.1 |
| | $2^{15}$ | 16 | 51.3 | 3.40 | 17.2 |

Table 4: Performance Comparison with PELTA [CMS+23]

**Key Generation for SPDZ Offline Phase.** Lastly, we demonstrate the efficacy of our ZKAoKs for MPHE in the offline phase of the SPDZ protocol [DPSZ12]. Specifically, the offline phase requires a maliciously secure protocol that generates joint encryption and relinearization keys for MPHE. The current state-of-the-art solution is the construction by Rotaru et al.[RST+22], which is based on another general MPC protocol [KOS16], built on oblivious transfer. We observe that the setup phase in Fig. 1, without automorphism keys, can also serve as a solution, as it generates valid joint encryption and joint relinearization keys in the presence of malicious adversaries. Therefore, we measure the performance of this solution to compare with [RST+22]. As shown in Table 5, our solution takes 91 seconds to generate and verify proofs for encryption and relinearization keys, while the previous solution takes 47 minutes to generate these keys via general MPC protocols under similar parameter settings. Thus, we conclude that our ZKAoKs can also be effectively utilized for the offline phase of the SPDZ protocol without relying on other general MPC protocols.

| | $\log q$ | $N$ | Time |
|---|---|---|---|
| [RST+22] | 380 | $2^{14}$ | 47 min |
| Ours | 429 | $2^{14}$ | 91 sec |

Table 5: Performance Comparison for Key Generation in SPDZ

## Acknowledgement

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. RS-2023-00211649) and the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No. RS-2024-00399491, Development of Privacy-Preserving Multiparty Computation Techniques for Secure Multiparty Data Integration).

## References

ACLS18. Sebastian Angel, Hao Chen, Kim Laine, and Srinath Setty. Pir with compressed queries and amortized query processing. In *2018 IEEE symposium on security and privacy (SP)*, pages 962–979. IEEE, 2018.

AJLA+12. Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold fhe. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 483–501. Springer, 2012.

ALS20. Thomas Attema, Vadim Lyubashevsky, and Gregor Seiler. Practical product proofs for lattice commitments. In *Annual International Cryptology Conference*, pages 470–499. Springer, 2020.

APS15. Martin R Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.

BBB+18. Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*, pages 315–334. IEEE, 2018.

BCFK21. Alexandre Bois, Ignacio Cascudo, Dario Fiore, and Dongwoo Kim. Flexible and efficient verifiable computation on encrypted data. In *IACR International Conference on Public-Key Cryptography*, pages 528–558. Springer, 2021.

BCOS20. Cecilia Boschini, Jan Camenisch, Max Ovsiankin, and Nicholas Spooner. Efficient post-quantum snarks for rsis and rlwe and their applications to privacy. In *Post-Quantum Cryptography: 11th International Conference, PQCrypto 2020, Paris, France, April 15–17, 2020, Proceedings 11*, pages 247–267. Springer, 2020.

BFS20. Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent snarks from dark compilers. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 677–706, 2020.

BLS19. Jonathan Bootle, Vadim Lyubashevsky, and Gregor Seiler. Algebraic techniques for short (er) exact lattice-based zero-knowledge proofs. In *Annual International Cryptology Conference*, pages 176–202. Springer, 2019.

Bra12. Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical gapsvp. In *Annual cryptology conference*, pages 868–886. Springer, 2012.

BSCR+19. Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P Ward. Aurora: Transparent succinct arguments for r1cs. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 103–128. Springer, 2019.

BSCS16. Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Proceedings, Part II, of the 14th International Conference on Theory of Cryptography-Volume 9986*, pages 31–60, 2016.

BV11. Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) lwe. In *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science*, pages 97–106. IEEE, 2011.

CBBZ23. Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 499–530. Springer, 2023.

CCP+24. Jung Hee Cheon, Hyeongmin Choe, Alain Passelègue, Damien Stehlé, and Elias Suvanto. Attacks against the ind-cpad security of exact fhe schemes. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 2505–2519, 2024.

CHLR18. Hao Chen, Zhicong Huang, Kim Laine, and Peter Rindal. Labeled psi from fully homomorphic encryption with malicious security. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1223–1237, 2018.

CHM+20. Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. Marlin: Preprocessing zksnarks with universal and updatable srs. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 738–768, 2020.

CKR+20. Hao Chen, Miran Kim, Ilya Razenshteyn, Dragos Rotaru, Yongsoo Song, and Sameer Wagh. Maliciously secure matrix multiplication with applications to private deep learning. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 31–59, 2020.

CMdG+21. Kelong Cong, Radames Cruz Moreno, Mariana Botelho da Gama, Wei Dai, Ilia Iliashenko, Kim Laine, and Michael Rosenberg. Labeled psi from homomorphic encryption with reduced computation and communication. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 1135–1150, 2021.

CMS+23. Sylvain Chatel, Christian Mouchet, Ali Utkan Sahin, Apostolos Pyrgelis, Carmela Troncoso, and Jean-Pierre Hubaux. Pelta-shielding multiparty-fhe against malicious adversaries. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 711–725, 2023.

CO17. Wutichai Chongchitmate and Rafail Ostrovsky. Circuit-private multi-key fhe. In *IACR International Workshop on Public Key Cryptography*, pages 241–270. Springer, 2017.

CSBB24. Marina Checri, Renaud Sirdey, Aymen Boudguiga, and Jean-Paul Bultel. On the practical cpa d security of "exact" and threshold fhe schemes and libraries. In *Annual International Cryptology Conference*, pages 3–33. Springer, 2024.

DPLS19. Rafaël Del Pino, Vadim Lyubashevsky, and Gregor Seiler. Short discrete log proofs for fhe and ring-lwe ciphertexts. In *IACR International Workshop on Public Key Cryptography*, pages 344–373. Springer, 2019.

DPSZ12. Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *Annual Cryptology Conference*, pages 643–662. Springer, 2012.

ENS20. Muhammed F Esgin, Ngoc Khanh Nguyen, and Gregor Seiler. Practical exact proofs from lattices: New techniques to exploit fully-splitting rings. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 259–288. Springer, 2020.

FNP20. Dario Fiore, Anca Nitulescu, and David Pointcheval. Boosting verifiable computation on encrypted data. In *Public-Key Cryptography–PKC 2020: 23rd IACR International Conference on Practice and Theory of Public-Key Cryptography, Edinburgh, UK, May 4–7, 2020, Proceedings, Part II 23*, pages 124–154. Springer, 2020.

FS86. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.

FV12. Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, 2012.

GIKV23. Robin Geelen, Ilia Iliashenko, Jiayi Kang, and Frederik Vercauteren. On polynomial functions modulo pe and faster bootstrapping for homomorphic encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 257–286. Springer, 2023.

GNSJ24. Qian Guo, Denis Nabokov, Elias Suvanto, and Thomas Johansson. Key recovery attacks on approximate homomorphic encryption with non-worst-case noise flooding countermeasures. In *Usenix Security*, 2024.

GNSV23. Chaya Ganesh, Anca Nitulescu, and Eduardo Soria-Vazquez. Rinocchio: Snarks for ring arithmetic. *Journal of Cryptology*, 36(4):41, 2023.

HPS19. Shai Halevi, Yuriy Polyakov, and Victor Shoup. An improved rns variant of the bfv homomorphic encryption scheme. In *Cryptographers' Track at the RSA Conference*, pages 83–105. Springer, 2019.

HSS24. Intak Hwang, Jinyeong Seo, and Yongsoo Song. Concretely efficient lattice-based polynomial commitment from standard assumptions. In *Annual International Cryptology Conference*, pages 414–448. Springer, 2024.

JPK+24. Jae Hyung Ju, Jaiyoung Park, Jongmin Kim, Minsik Kang, Donghwan Kim, Jung Hee Cheon, and Jung Ho Ahn. Neujeans: Private neural network inference with joint optimization of convolution and fhe bootstrapping. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 4361–4375, 2024.

KKL+23. Taechan Kim, Hyesun Kwak, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. Asymptotically faster multi-key homomorphic encryption from homomorphic gadget decomposition. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 726–740, 2023.

KLSW24. Hyesun Kwak, Dongwon Lee, Yongsoo Song, and Sameer Wagh. A general framework of homomorphic encryption for multiple parties with non-interactive key-aggregation. In *International Conference on Applied Cryptography and Network Security*, pages 403–430. Springer, 2024.

KOS16. Marcel Keller, Emmanuela Orsini, and Peter Scholl. Mascot: faster malicious arithmetic secure computation with oblivious transfer. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 830–842, 2016.

KSS24. Jaehyung Kim, Jinyeong Seo, and Yongsoo Song. Simpler and faster bfv bootstrapping for arbitrary plaintext modulus from ckks. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 2535–2546, 2024.

LATV12. Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In *Proceedings of the forty-fourth annual ACM symposium on Theory of computing*, pages 1219–1234, 2012.

LLL$^+$22. Eunsang Lee, Joon-Woo Lee, Junghyun Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and Woosuk Choi. Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In *International Conference on Machine Learning*, pages 12403–12422. PMLR, 2022.

LM21. Baiyu Li and Daniele Micciancio. On the security of homomorphic encryption on approximate numbers. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 648–677. Springer, 2021.

LNS20. Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Gregor Seiler. Practical lattice-based zero-knowledge proofs for integer relations. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*, pages 1051–1070, 2020.

LNSW13. San Ling, Khoa Nguyen, Damien Stehlé, and Huaxiong Wang. Improved zero-knowledge proofs of knowledge for the isis problem, and applications. In *International workshop on public key cryptography*, pages 107–124. Springer, 2013.

LPR10. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In *Advances in Cryptology–EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, French Riviera, May 30–June 3, 2010. Proceedings 29*, pages 1–23. Springer, 2010.

LT22. Zeyu Liu and Eran Tromer. Oblivious message retrieval. In *Annual International Cryptology Conference*, pages 753–783. Springer, 2022.

LTW24. Zeyu Liu, Eran Tromer, and Yunhao Wang. Group oblivious message retrieval. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4367–4385. IEEE, 2024.

LW24. Zeyu Liu and Yunhao Wang. Relaxed functional bootstrapping: A new perspective on bgv/bfv bootstrapping. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 208–240, 2024.

MCPT24. Christian Mouchet, Sylvain Chatel, Apostolos Pyrgelis, and Carmela Troncoso. Helium: Scalable mpc among lightweight participants and under churn. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3038–3052, 2024.

MTPBH21. Christian Mouchet, Juan Troncoso-Pastoriza, Jean-Philippe Bossuat, and Jean-Pierre Hubaux. Multiparty homomorphic encryption from ring-learning-with-errors. *Proceedings on Privacy Enhancing Technologies*, 2021(4):291–311, 2021.

MV24. Daniele Micciancio and Vinod Vaikuntanathan. Sok: Learning with errors, circular security, and fully homomorphic encryption. In *IACR International Conference on Public-Key Cryptography*, pages 291–321. Springer, 2024.

MW16. Pratyay Mukherjee and Daniel Wichs. Two round multiparty computation via multi-key fhe. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 735–763. Springer, 2016.

MW22. Samir Jordan Menon and David J Wu. Spiral: Fast, high-rate single-server pir via fhe composition. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 930–947. IEEE, 2022.

RST+22. Dragos Rotaru, Nigel P Smart, Titouan Tanguy, Frederik Vercauteren, and Tim Wood. Actively secure setup for spdz. *Journal of Cryptology*, 35(1):5, 2022.

# A PIOP Compilation

A polynomial commitment scheme (PCS) is a class of commitment scheme that takes polynomials as messages and allows the evaluation of committed polynomials. Below, we define a polynomial commitment scheme for univariate polynomials, adapted from [CHM+20].

**Definition 3 (Polynomial Commitment).** *A polynomial commitment* $\mathtt{PC}$ *consists of the following PPT algorithms.*

- $\underline{\mathtt{PC.Setup}}(1^\lambda, D) \to \mathsf{ck}$*: Given a security parameter* $\lambda$ *and a global polynomial degree upper bound* $D$*, it generates a commitment key* $\mathsf{ck}$*.*
- $\underline{\mathtt{PC.Com}}(\mathsf{ck}, d, \boldsymbol{f}) \to (c, \delta)$*: Given a polynomial* $\boldsymbol{f} \in \mathbb{F}[X]$ *with degree* $< d$*, it generates a commitment* $c$ *and an opening hint* $\delta$*.*
- $\underline{\mathtt{PC.Open}}(\mathsf{ck}, c, d, \boldsymbol{f}, \delta) \to b$*: Given a commitment* $c$*, a polynomial* $\boldsymbol{f}$ *with degree* $< d$*, and an opening hint* $\delta$*, it outputs* $0$ *or* $1$*.*
- $\underline{\mathtt{PC.Eval}}(\mathsf{ck}, x, d, \boldsymbol{f}, \delta) \to (y, \rho)$*: Given an evaluation point* $x \in \mathbb{F}$ *and an opening hint* $\delta$*, it returns an evaluation result* $y$*, and an evaluation proof* $\rho$*.*
- $\underline{\mathtt{PC.Check}}(\mathsf{ck}, c, d, x, y, \rho) \to b$*: Given a commitment* $c$*, a degree upper bound* $d$*, an evaluation point* $x$*, an evaluation result* $y$*, and an evaluation proof* $\rho$*, it outputs* $0$ *or* $1$*.*

$\mathtt{PC}$ *is called a polynomial commitment scheme if it satisfies the following properties.*

- ***Correctness****: For every polynomial* $\boldsymbol{f} \in \mathbb{F}[X]$ *with a degree upper bound* $d \leq D$ *and every point* $x \in \mathbb{F}$*, the following holds.*

$$\Pr\left[\begin{array}{r|c} \mathtt{PC.Open}(\mathsf{ck}, c, d, \boldsymbol{f}, \delta) = 1 \wedge & \mathsf{ck} \leftarrow \mathtt{PC.Setup}(1^\lambda, D) \\ \mathtt{PC.Check}(\mathsf{ck}, c, d, x, \boldsymbol{f}(x), \rho) = 1 & (c, \delta) \leftarrow \mathtt{PC.Com}(\mathsf{ck}, d, \boldsymbol{f}) \\ & (y, \rho) \leftarrow \mathtt{PC.Eval}(x, \delta) \end{array}\right] \geq 1 - \mathsf{negl}(\lambda)$$

- ***Extractability****: For every PPT adversary* $\mathtt{A}$*, there exists a PPT extractor* $\mathtt{E}$ *such that for all randomness* $\mathsf{r}$*, the following holds.*

$$\Pr\left[\begin{array}{r|l} \mathtt{PC.Check}(\mathsf{ck}, c, d, x, y, \rho) = 1 \wedge & \mathsf{ck} \leftarrow \mathtt{PC.Setup}(1^\lambda, D) \\ \left(\mathtt{PC.Open}(\mathsf{ck}, c, d, \boldsymbol{f}, \delta) = 0 \vee\right. & (c, d, x, y, \rho) \leftarrow \mathtt{A}(\mathsf{ck}, \mathsf{r}) \\ \left. y \neq \boldsymbol{f}(x)\right) & (\boldsymbol{f}, \delta) \leftarrow \mathtt{E}(\mathsf{ck}, \mathsf{r}) \end{array}\right] \leq \mathsf{negl}(\lambda)$$

– ***Binding****: For every PPT adversary* $\mathtt{A}$*, the following holds.*

$$\Pr\left[\begin{array}{r|r} \mathtt{PC.Open}(\mathsf{ck}, c, d, \boldsymbol{f}, \delta) = 1 \wedge & \mathsf{ck} \leftarrow \mathtt{PC.Setup}(1^\lambda, D) \\ \mathtt{PC.Open}(\mathsf{ck}, c, d, \boldsymbol{f}', \delta') = 1 \wedge & (c, d, \boldsymbol{f}, \boldsymbol{f}', \delta, \delta') \leftarrow \mathtt{A}(\mathsf{ck}) \\ \boldsymbol{f} \neq \boldsymbol{f}' & \end{array}\right] \leq \mathsf{negl}(\lambda)$$

$\mathtt{PC}$ *is called hiding if the following property holds.*

– ***Hiding****: For every PPT adversary* $\mathtt{A} = (\mathtt{A}_1, \mathtt{A}_2)$*, there exists a PPT simulator* $\mathtt{S}$ *such that the following holds.*

$$\left|\Pr\left[\begin{array}{r|r} \mathtt{A}_2(\mathsf{ck}, c, \rho) = 1 \wedge & \mathsf{ck} \leftarrow \mathtt{PC.Setup}(1^\lambda, D) \\ \mathtt{PC.Check}(\mathsf{ck}, c, d, x, \boldsymbol{f}(x), \rho) = 1 & (d, \boldsymbol{f}, x) \leftarrow \mathtt{A}_1(\mathsf{ck}) \\ & (c, \rho) \leftarrow \mathtt{S}(\mathsf{ck}, x, \boldsymbol{f}(x)) \end{array}\right] - \Pr\left[\begin{array}{r|r} \mathtt{A}_2(\mathsf{ck}, c, \rho) = 1 \wedge & \mathsf{ck} \leftarrow \mathtt{PC.Setup}(1^\lambda, D) \\ \mathtt{PC.Check}(\mathsf{ck}, c, d, x, \boldsymbol{f}(x), \rho) = 1 & (d, \boldsymbol{f}, x) \leftarrow \mathtt{A}_1(\mathsf{ck}) \\ & (c, \delta) \leftarrow \mathtt{PC.Com}(\mathsf{ck}, d, \boldsymbol{f}) \\ & (y, \rho) \leftarrow \mathtt{PC.Eval}(x, \delta) \end{array}\right]\right| \leq \mathsf{negl}(\lambda)$$

**Theorem 7 ([CHM+20, Theorem 8.1]).** *Let* $\Pi$ *be a PIOP for a relation* $\mathsf{R}$ *and* $\mathtt{PC}$ *be a polynomial commitment scheme. Then, there exists a public coin argument of knowledge* $\Pi'$ *for* $\mathsf{R}$ *with the following complexity.*

– ***Prover complexity****: The sum of the runtime of the PIOP prover, the time to commit polynomials in* $\mathtt{PC}$*, and the time to produce evaluation proofs for oracle queries in* $\mathtt{PC}$*.*
– ***Verifier complexity****: The sum of the runtime of the PIOP verifier, the time to verify evaluation proofs in* $\mathtt{PC}$*.*
– ***Proof size****: The sum of the messages from the PIOP verifier, commitments size in* $\mathtt{PC}$*, and evaluation proof size in* $\mathtt{PC}$*.*

*Additionally, if* $\Pi$ *is HVZK and* $\mathtt{PC}$ *is hiding, then* $\Pi'$ *is HVZK.*

# B Deferred Protocols

$$\Pi_{\mathtt{RC}}(\vec{\mathtt{C}}; [\![\hat{\boldsymbol{a}}_0]\!], \ldots, [\![\hat{\boldsymbol{a}}_{n-1}]\!])$$

**Instance**: $n$-ary polynomial $\vec{\mathtt{C}} \in (\mathbb{Z}_p^N)[\vec{X}_0, \ldots, \vec{X}_{n-1}]$ of degree $d$, and polynomial oracles $([\![\hat{\boldsymbol{a}}_i]\!], L)$ for $0 \le i < n$.
**Witness**: vectors $\vec{a}_i = \mathtt{Dcd}(\hat{\boldsymbol{a}}_i)$ for $0 \le i < n$.
**Statement**: $\vec{\mathtt{C}}(\vec{a}_0, \ldots, \vec{a}_{n-1}) = 0$.

1. Both $\mathtt{P}$ and $\mathtt{V}$ compute $\mathtt{C} \in (\mathbb{Z}_p[X])[\boldsymbol{X}_0, \ldots, \boldsymbol{X}_{n-1}]$ of degree $d$, which is obtained by applying $\mathtt{Ecd}$ to each coefficient of $\vec{\mathtt{C}}$.
2. The prover $\mathtt{P}$ computes the quotient polynomial $\boldsymbol{q} = \mathtt{C}(\hat{\boldsymbol{a}}_0, \ldots, \hat{\boldsymbol{a}}_{n-1})/\boldsymbol{z}_{\mathbb{H}}$, and sends a polynomial oracle $([\![\boldsymbol{q}]\!], d(L-1))$ to $\mathtt{V}$.
3. The verifier $\mathtt{V}$ gets evaluations $\hat{\boldsymbol{a}}_i(\alpha)$ and $\boldsymbol{q}(\alpha)$ by accessing polynomial oracles $[\![\hat{\boldsymbol{a}}_i]\!]$, $[\![\boldsymbol{q}]\!]$ at a random point $\alpha \leftarrow \mathcal{U}(\mathbb{Z}_p \backslash \mathbb{H})$. Then, $\mathtt{V}$ checks whether the following holds.
$$\boldsymbol{q}(\alpha) \cdot \boldsymbol{z}_{\mathbb{H}}(\alpha) = \big(\mathtt{C}(\hat{\boldsymbol{a}}_0, \ldots, \hat{\boldsymbol{a}}_{n-1})\big)(\alpha)$$

Fig. 7: PIOP for arithmetic constraints

$$\Pi_{\mathtt{LC}}(M; [\![\hat{\boldsymbol{a}}_0]\!], \ldots, [\![\hat{\boldsymbol{a}}_{n-1}]\!], [\![\hat{\boldsymbol{b}}_0]\!], \ldots, [\![\hat{\boldsymbol{b}}_{n-1}]\!])$$

**Instance**: matrix $M \in \mathbb{Z}_p^{N \times N}$, and polynomial oracles $([\![\hat{\boldsymbol{a}}_i]\!], L)$ and $([\![\hat{\boldsymbol{b}}_i]\!], L)$ for $0 \le i < n$.
**Witness**: vectors $\vec{a}_i = \mathtt{Dcd}(\hat{\boldsymbol{a}}_i)$ and $\vec{b}_i = \mathtt{Dcd}(\hat{\boldsymbol{b}}_i)$ for $0 \le i < n$.
**Statement**: $\vec{b}_i = M\vec{a}_i$ for $0 \le i < n$.

1. The prover $\mathtt{P}$ samples a random polynomial $\boldsymbol{g} \leftarrow \mathcal{U}(\mathbb{Z}_p)^{L+N}$, computes the summation $\mu = \sum_{h \in \mathbb{H}} \boldsymbol{g}(h)$, and sends a polynomial oracle $([\![\boldsymbol{g}]\!], L+N-1)$ and $\mu$ to the verifier $\mathtt{V}$.
2. The verifier $\mathtt{V}$ sends random points $\beta, v \leftarrow \mathcal{U}(\mathbb{Z}_p \setminus \mathbb{H})$.
3. The prover $\mathtt{P}$ computes $\boldsymbol{q}$ and $\boldsymbol{r}$, which satisfy the following for the polynomials $\boldsymbol{v}, \boldsymbol{w} \in \mathbb{Z}_p^{<N}[X]$ such that $\vec{v} = (1, v, \ldots, v^{N-1})$, $\boldsymbol{v} = \mathtt{Ecd}(\vec{v})$, and $\boldsymbol{w} = \mathtt{Ecd}(M^\top \vec{v})$.
$$\begin{aligned}&\boldsymbol{g}(X) + \sum_{i=0}^{n-1} \beta^{i+1} \cdot \Big(\hat{\boldsymbol{a}}_i(X)\boldsymbol{w}(X) - \hat{\boldsymbol{b}}_i(X)\boldsymbol{v}(X)\Big)\\ &= \boldsymbol{q}(X) \cdot \boldsymbol{z}_{\mathbb{H}}(X) + \boldsymbol{r}(X) \cdot X + N^{-1} \cdot \mu\end{aligned}$$
Then, $\mathtt{P}$ sends polynomial oracles $([\![\boldsymbol{q}]\!], L-1)$ and $([\![\boldsymbol{r}]\!], N-1)$ to the verifier $\mathtt{V}$.
4. The verifier $\mathtt{V}$ gets evaluations $\hat{\boldsymbol{a}}_i(\alpha)$, $\hat{\boldsymbol{b}}_i(\alpha)$, $\boldsymbol{g}(\alpha)$, $\boldsymbol{q}(\alpha)$, $\boldsymbol{r}(\alpha)$ by accessing polynomial oracles $[\![\hat{\boldsymbol{a}}_i]\!]$, $[\![\hat{\boldsymbol{b}}_i]\!]$, $[\![\boldsymbol{g}]\!]$, $[\![\boldsymbol{q}]\!]$, and $[\![\boldsymbol{r}]\!]$ at a random point $\alpha \leftarrow \mathcal{U}(\mathbb{Z}_p \setminus \mathbb{H})$. Then, $\mathtt{V}$ checks whether
$$\begin{aligned}&\boldsymbol{g}(\alpha) + \sum_{i=0}^{n-1} \beta^{i+1} \cdot \Big(\hat{\boldsymbol{a}}_i(\alpha)\boldsymbol{w}(\alpha) - \hat{\boldsymbol{b}}_i(\alpha)\boldsymbol{v}(\alpha)\Big)\\ &= \boldsymbol{q}(\alpha) \cdot \boldsymbol{z}_{\mathbb{H}}(\alpha) + \boldsymbol{r}(\alpha) \cdot \alpha + N^{-1} \cdot \mu\end{aligned}$$

Fig. 8: PIOP for linear relation check

# C Deferred Proofs

## C.1 Proof sketch of Theorem 1 and Theorem 2

The main idea behind the two PIOPs, $\Pi_{\mathtt{RC}}$ and $\Pi_{\mathtt{LC}}$, is to convert algebraic constraints into polynomial equations, such as $\boldsymbol{f}(x) = 0$ for all $x \in \mathbb{H}$, which is equivalent to $\boldsymbol{f}$ being divisible by the vanishing polynomial $\boldsymbol{z}_{\mathbb{H}}$. To demonstrate divisibility, the prover $\mathtt{P}$ sends the quotient polynomial $\boldsymbol{q}$, and the verifier $\mathtt{V}$ checks the polynomial equation $\boldsymbol{f}(X) = \boldsymbol{q}(X) \cdot \boldsymbol{z}_{\mathbb{H}}(X)$ at a randomly chosen point. For the algebraic-to-polynomial conversion to be correct, the following condition must hold.

Regarding soundness, if the algebraic constraints are unsatisfiable, the resulting polynomial equation will also be unsatisfiable. For the protocol to be accepted, $\mathtt{P}$ must provide a polynomial oracle $[\![\boldsymbol{q}']\!]$ that satisfies $\boldsymbol{f}(\alpha) = \boldsymbol{q}'(\alpha) \cdot \boldsymbol{z}_{\mathbb{H}}(\alpha)$, even if $\boldsymbol{f}(X) \neq \boldsymbol{q}'(X) \cdot \boldsymbol{z}_{\mathbb{H}}(X)$, where $\alpha$ is a query point chosen by $\mathtt{V}$. The probability of this succeeding is at most $O(\deg(\boldsymbol{f})/|C|)$, which is negligible when a challenge set $|C|$ is exponentially large. Thus, soundness is achieved. Furthermore, because a sound PIOP satisfies knowledge soundness (Lemma 2.3, [CBBZ23]), we can conclude that both $\Pi_{\mathtt{RC}}$ and $\Pi_{\mathtt{LC}}$ provide knowledge soundness.

To ensure HVZK, the prover $\mathtt{P}$ applies randomized encoding to the witness vectors. Due to bounded independence, the distributions of the polynomial oracle responses from the randomized encoded polynomial $[\![\mathtt{REcd}(\vec{a})]\!]$ and from a randomly sampled polynomial $[\![\boldsymbol{p}]\!]$ are indistinguishable for up to $L - N$ queries. Since the query complexity of PIOPs $\Pi_{\mathtt{RC}}$ and $\Pi_{\mathtt{LC}}$ is 1, we can use any $L \leq N+1$.

Hereafter, we focus on complexity analysis for PIOPs: soundness error $\epsilon_{\mathtt{PIOP}}$, prover/verifier time $\mathtt{Ptime}_{\mathtt{PIOP}}/\mathtt{Vtime}_{\mathtt{PIOP}}$, query complexity $Q_{\mathtt{PIOP}}$, size of polynomial oracle $\mathtt{Osize}_{\mathtt{PIOP}}$, and size of witness $\mathtt{Wsize}_{\mathtt{PIOP}}$.

## C.2 Analysis of $\Pi_{\mathtt{RC}}$ in Fig. 7 (Proof of Theorem 1)

In this section, we provide complexity analysis of row check PIOP $\Pi_{\mathtt{RC}}$ for $n$ variable polynomial of degree $d$ with $m$ non-zero entries.

- **Soundness Error,** $\epsilon_{\mathtt{RC}}(m, n, d)$:
  Instead of verifying the entire polynomial equation involving $\hat{\boldsymbol{a}}_i, \boldsymbol{q}$, the verifier accesses polynomial oracles at a random point $\alpha \leftarrow \mathcal{U}(\mathbb{Z}_p \setminus \mathbb{H})$ and checks the verifier equation involving evaluations returned by these oracles. A soundness error occurs if the verifier equation holds while the original polynomial equation does not. Since the degree of the polynomial equation is at most $dN$, the soundness error of $\Pi_{\mathtt{RC}}$ is can be bounded using the Schwartz-Zippel lemma as follows: $O\left(\frac{dN}{|\mathbb{Z}_p \setminus \mathbb{H}|}\right) = O\left(\frac{dN}{p-N}\right)$.
- **Prover time,** $\mathtt{Ptime}_{\mathtt{RC}}(m, n, d)$:
  The prover's actions consist of the following two steps.
  1. Encoding coefficient vectors of $\vec{\mathtt{C}}$ into $\mathtt{C}$:
     $\vec{\mathtt{C}}$ consists of $m$ coefficient vectors, each of length $N$, and the total cost is equivalent to the sum of encoding cost for each coefficient vector. Thus, the total encoding cost is $O(mN \log N)$

2. Compute the quotient polynomial $\boldsymbol{q}$:
   To compute $\boldsymbol{q}$, the prover first computes the NTT forms of $\hat{\boldsymbol{a}}_i$ for polynomial multiplication, that requires $O(nN \log N)$ operation. Next, the prover computes the univariate polynomial $\mathbf{C}(\hat{\boldsymbol{a}}_0, \ldots, \hat{\boldsymbol{a}}_{n-1})(X)$, performing vector-wise multiplication of the NTT forms of the univariate polynomial, followed by an inverse NTT. The computational costs for these steps are $O(mdN)$ and $O(dN \log(dN))$, respectively. Finally, the prover divides the univariate polynomial by $\boldsymbol{z}_{\mathbb{H}}$, which has a complexity of $O(dN)$.

Therefore, the total prover time complexity is $O((n+m)N \log N + mdN + dN \log(dN))$

– **Verifier time,** $\mathtt{Vtime}_{\mathtt{RC}}(m, n, d)$:
The verifier's action consists of the following two steps.
1. Encoding coefficient vectors of $\vec{\mathbf{C}}$ into $\mathbf{C}$:
   In the same way in the prover computation, the complexity is $O(mN \log N)$
2. Verifying $\boldsymbol{q}(\alpha) \cdot \boldsymbol{z}_{\mathbb{H}}(\alpha) = \big(\mathbf{C}(\hat{\boldsymbol{a}}_0, \ldots, \hat{\boldsymbol{a}}_{n-1})\big)(\alpha)$:
   To check the verifier equation, the verifier first compute $\boldsymbol{z}_{\mathbb{H}}(\alpha)$, which has a complexity of $O(\log N)$. Next, the verifier evaluates $\mathbf{C}(\hat{\boldsymbol{a}}_0, \ldots, \hat{\boldsymbol{a}}_{n-1})$ using oracle responses $\hat{\boldsymbol{a}}_0(\alpha), \ldots, \hat{\boldsymbol{a}}_{n-1}(\alpha)$. The computational costs for the evaluation is $O(mN + md)$

Therefore, total verifier time complexity is $O(mN \log N + dm)$.

– **Query Complexity**, $Q_{\mathtt{RC}}(m, n, d)$: The verifier access to $n+1$ polynomial oracles $[\![\boldsymbol{q}]\!]$ and $[\![\hat{\boldsymbol{a}}_i]\!]$ at a point $\alpha$. Therefore, the query complexity $Q_{\mathtt{RC}}(m, n, d)$ is $n+1$.

– **Oracle Size**, $\mathtt{Osize}_{\mathtt{RC}}(m, n, d)$: The degrees of the polynomial oracle $[\![\boldsymbol{q}]\!]$ is $\leq dL$. Thus, the size of proof oracles $\mathtt{Osize}_{\mathtt{RC}}(m, n, d)$ is $\leq dL$ elements in $\mathbb{Z}_p$.

– **Witness Size**, $\mathtt{Wsize}_{\mathtt{RC}}(m, n, d)$: The witness consists of $n$ distinct vectors in $\mathbb{Z}_p^N$. Thus, the size of witness $\mathtt{Wsize}_{\mathtt{RC}}(m, n, d)$ is $nN$ elements in $\mathbb{Z}_p$.

### C.3 Analysis of $\Pi_{\mathtt{LC}}$ in Fig. 8 (Proof of Theorem 2)

In this section, we provide complexity analysis of linear check PIOP $\Pi_{\mathtt{LC}}$ for $n$ pairs of witness vectors $(\vec{a}_i, \vec{b}_i)_{i=0}^{n-1}$ with $k$ non-zero entries of the matrix $M$.

– **Soundness Error,** $\epsilon_{\mathtt{LC}}(n, k)$:
Instead of verifying the entire polynomial equation involving $\hat{\boldsymbol{a}}_i, \hat{\boldsymbol{b}}_i, \boldsymbol{q}, \boldsymbol{g}, \boldsymbol{r}$, the verifier accesses polynomial oracles at a random point $\alpha \leftarrow \mathcal{U}(\mathbb{Z}_p \setminus \mathbb{H})$ and verifies the evaluations returned by these oracles. A soundness error occurs if the verifier's equation holds while the original polynomial equation does not. We consider two cases: errors in batching polynomials and errors in evaluation checks.
1. Error in batching polynomials: The batched polynomial equation
$$\sum_{i=0}^{n-1} \beta^{i+1} \cdot \Big(\hat{\boldsymbol{a}}_i(X)\boldsymbol{w}(X) - \hat{\boldsymbol{b}}_i(X)\boldsymbol{v}(X)\Big) = 0$$
may hold even though the individual polynomial equation $\hat{\boldsymbol{a}}_i(X)\boldsymbol{w}(X) - \hat{\boldsymbol{b}}_i(X)\boldsymbol{v}(X) = 0$ does not hold for some $i = 0, \ldots, n-1$. In this case, the soundness error is $O\left(\frac{n}{p-N}\right)$.

2. Error in evaluation checks: The verifier's equation, whose degree is $O(N)$, may hold even though the underlying polynomial equation does not. By the Schwartz-Zippel lemma, the soundness error in this case is $O\left(\frac{N}{p-N}\right)$.

Therefore, the total soundness error is $O\left(\frac{N+n}{p-N}\right)$.

– **Prover time,** $\mathtt{Ptime}_{\mathtt{LC}}(n,k)$:
The prover's actions consist of the following two steps.
1. Generating $\boldsymbol{g}$ and compute $\mu$:
The prover generates $\boldsymbol{g}$ and then compute $\mu$. Using the algebraic trick used in univariate sumcheck, it can be computed within the $O(N)$ operation.
2. Computes $\boldsymbol{q}, \boldsymbol{r}$:
To compute $\boldsymbol{q}$, the prover first construct $\boldsymbol{v}$ and $\boldsymbol{w}$ with encoding and matrix multiplication. In this step, the prover complexity is $O(k + N \log N)$.
Next, the prover aggregates $2n$ polynomials $\hat{\boldsymbol{a}}_i$ and $\hat{\boldsymbol{b}}_i$, whose complexity is $O(nN \log N)$. Finally, the prover computes a quotient polynomial $\boldsymbol{q}$ and a residue polynomial $\boldsymbol{r}$ within $O(N \log N)$ computation. Totally, the prover complexity for construct $\boldsymbol{q}, \boldsymbol{r}$ is $O(nN \log N)$.

Therefore, the total cost for prover is $O(k + nN \log N)$.

– **Verifier time,** $\mathtt{Vtime}_{\mathtt{LC}}(n,k)$:
The verifier's action consists of the following two steps.
1. Evaluate $\boldsymbol{v}(\alpha)$ and $\boldsymbol{w}(\alpha)$:
The verifier constructs two polynomials, $\boldsymbol{v}$ and $\boldsymbol{v}$, and evaluates them at a point $\alpha$. The complexity of constructing $\boldsymbol{v}$ and $\boldsymbol{w}$ is $O(k + N \log N)$, and the complexity of evaluating them is $O(N)$. Thus, the total complexity for computing $\boldsymbol{v}(\alpha)$ and $\boldsymbol{w}(\alpha)$ is $O(k + N \log N)$.
2. Verifier Equation:
To check the verifier equation, the verifier first compute $\boldsymbol{z}_{\mathbb{H}}(\alpha)$, which has a complexity of $O(\log N)$. Next, the verifier evaluates $\mathtt{C}$, $(\hat{\boldsymbol{a}}_0, \ldots, \hat{\boldsymbol{a}}_{n-1})$, $(\hat{\boldsymbol{b}}_0, \ldots, \hat{\boldsymbol{b}}_{n-1})$ using oracle responses $\hat{\boldsymbol{a}}_0(\alpha), \ldots, \hat{\boldsymbol{a}}_{n-1}(\alpha)$. The computational costs is $O(n + \log N)$

Therefore, total verifier time is $O(n + k + N \log N)$.

– **Query Complexity**, $Q_{\mathtt{LC}}(n,k)$: The verifier accesses to $2n + 3$ polynomial oracles $[\![\hat{\boldsymbol{a}}_i]\!]$, $[\![\hat{\boldsymbol{b}}_i]\!]$, $[\![\boldsymbol{g}]\!]$, $[\![\boldsymbol{q}]\!]$, and $[\![\boldsymbol{r}]\!]$ at a point $\alpha$. Therefore, the query complexity $Q_{\mathtt{LC}}(n,k)$ is $2n + 3$.

– **Oracle Size**,$\mathtt{Osize}_{\mathtt{LC}}(n,k)$: The sum of degrees of polynomial oracles $[\![\boldsymbol{g}]\!]$, $[\![\boldsymbol{q}]\!]$, and $[\![\boldsymbol{r}]\!]$ is $\leq 2L + 2N$. Thus, the size of polynomial oracles $\mathtt{Osize}_{\mathtt{LC}}(n,k)$ is $\leq 2L + 2N$ elements in $\mathbb{Z}_p$.

– **Witness Size**, $\mathtt{Wsize}_{\mathtt{LC}}(n,k)$: The witness consists of $2n$ vectors in $\mathbb{Z}_p^N$. Then, the size of witness $\mathtt{Wsize}_{\mathtt{LC}}(n,k)$ is $2nN$ elements in $\mathbb{Z}_p$.

### C.4 Analysis of $\Pi_{\mathtt{NC}}$ in Fig. 3 (Proof of Theorem 3)

In this section, we provide complexity analysis of norm check PIOP $\Pi_{\mathtt{NC}}$ for $n$ witness $(\vec{a}_i)_{i=0}^{n-1}$ with $k$ length norm bound $B$, i.e $k = \lfloor \log B \rfloor + 1$.

Since the norm check PIOP $\Pi_{\mathtt{NC}}$ reduced to a row check PIOP $\Pi_{\mathtt{RC}}$, the complexity and security properties of $\Pi_{\mathtt{NC}}$ follow from those of $\Pi_{\mathtt{RC}}$.

By the completeness, knowledge soundness, and HVZK of $\Pi_{\mathtt{RC}}$ in Theorem 1, $\Pi_{\mathtt{NC}}$ achieves perfect completeness, knowledge soundness, and HVZK.
Next, we consider the complexity analysis of the norm check PIOP $\Pi_{\mathtt{NC}}$.

- **Soundness Error,** $\epsilon_{\mathtt{NC}}(n,k)$:
  The soundness error arising from batching polynomials using random points $\beta$ and $\gamma$ is $O\left(\frac{nk}{N-p}\right)$. Additionally, the soundness error induced by the invocation of $\Pi_{\mathtt{RC}}$ is $O\left(\frac{3N}{p-N}\right)$ because the degree of multivariate polynomial is at most 3. Thus, the total soundness error is $O\left(\frac{N+nk}{p-N}\right)$.
- **Prover time,** $\mathtt{Ptime}_{\mathtt{NC}}(n,k)$:
  The prover's actions consist of the following three steps.
  1. Decompose norm bound, $B \to (B_0, \dots B_{k-1})$:
     The decomposition consists of $O(k)$ field operations.
  2. Decompose witness vectors, $\vec{a}_i \to (\vec{a}_{i,0}, \dots, \vec{a}_{i,k-1})$:
     For each witness vector $\vec{a}_i$, the decomposition requires $O(kN)$ operation. Thus, the total decomposition cost is $O(nkN)$.
  3. One $\Pi_{\mathtt{RC}}$:
     The prover time $\mathtt{Ptime}_{\mathtt{RC}}(n(1+3k), n(k+1), 3)$ of row check PIOP for a $n(k+1)$-variable polynomial with degree $d=3$ and $m = n(1+3k)$ non-zero terms is $O(nkN \log N)$.

  The total prover time is $O(nkN \log N)$.
- **Verifier time,** $\mathtt{Vtime}_{\mathtt{NC}}(n,k)$:
  The verifier's actions consist of the following two steps.
  1. Decompose norm bound, $B \to (B_0, \dots B_{k-1})$:
     The decomposition consists of $O(k)$ field operations.
  2. One $\Pi_{\mathtt{RC}}$:
     The verifier time $\mathtt{Vtime}_{\mathtt{RC}}(n(1+3k), n(k+1), 3)$ of row check PIOP for a $n(k+1)$-variable polynomial with degree $d=3$ and $m = n(1+3k)$ non-zero terms is $O(nkN \log N)$.

  The total verifier time is $O(nkN \log N)$.
- **Query Complexity**, $Q_{\mathtt{NC}}(n,k)$: The verifier queries at $nk+n+1$ polynomial oracles: $[\![\hat{\boldsymbol{a}}_i]\!]$ and $[\![\hat{\boldsymbol{a}}_{i,j}]\!]$ for all $i = 0, \dots n-1$ and $j = 0, \dots, k-1$, and quotient polynomial $\boldsymbol{q}$ from the $\Pi_{\mathtt{RC}}$ PIOP. Therefore, the query complexity $Q_{\mathtt{NC}}(n,k)$ is $nk+n+1$.
- **Oracle Size**, $\mathtt{Osize}_{\mathtt{NC}}(n,k)$: In step 2, the sum of oracle size is $nkL$. In step 4, the oracle size for norm check is $\leq 3L$. Thus, the total oracle size $\mathtt{Osize}_{\mathtt{NC}}(n,k)$ is $\leq (nk+3)L$ element in $\mathbb{Z}_p$.
- **Witness Size**, $\mathtt{Wsize}_{\mathtt{NC}}(n,k)$: The witness consists of $n$ vectors in $\mathbb{Z}_p^N$. Thus, the size of witness $\mathtt{Wsize}_{\mathtt{NC}}(n,k)$ is $nN$ elements in $\mathbb{Z}_p$.

### C.5 Analysis of $\Pi_{\mathtt{KG}}$ in Fig. 4 (Proof of Theorem 4)

Let us consider the structure of $\Pi_{\mathtt{KG}}$. The PIOP $\Pi_{\mathtt{KG}}$ is reduced to multiple PIOPs as follows: 1 norm check PIOP $\Pi_{\mathtt{NC}}$, 3 linear check PIOP $\Pi_{\mathtt{LC}}$, and 3 row

check PIOP $\Pi_{\mathtt{RC}}$. Concretely, in step 3, $\Pi_{\mathtt{KG}}$ invokes norm check PIOP for $5\ell+4$ witness vectors, $k$-length norm bound $B$. In step 4, it invokes linear check PIOP for $5\ell+4$ pairs of witnesses with NTT matrix $T$. In step 5, it invokes rowcheck PIOP for a degree-1 2 variable polynomial with 3 non-zero terms. In step 6, it invockes rowcheck PIOP for a degree-1 $3\ell+2$ variable polynomial with $3\ell+3$ non-zero terms. In step 7, it invokes two linearcheck PIOP for a pair of witnesses with permutation matrices $P_0$ and $P_1$, respectively. After then, it also invockes a rowcheck PIOP for degree-1 $2\ell+3$ variable polynomial with $2\ell+4$ non-zero terms.

By the completeness, knowledge soundness, and HVZK properties of $\Pi_{\mathtt{LC}}$ (Theorem 2), $\Pi_{\mathtt{RC}}$ (Theorem 1), and $\Pi_{\mathtt{NC}}$ (Theorem 3), the PIOP $\Pi_{\mathtt{KG}}$ in Fig. 4 achieves perfect completeness, knowledge soundness, and HVZK.
We consider the complexity analysis of the PIOP $\Pi_{\mathtt{KG}}$.

– **Soundness Error,** $\epsilon_{\mathtt{KG}}$:
The sum of each soundness from PIOP invocations is as follows:

$$\epsilon_{\mathtt{NC}}(5\ell+4,k)+\epsilon_{\mathtt{LC}}(5\ell+4,N^2)+2*\epsilon_{\mathtt{LC}}(1,N)+\epsilon_{\mathtt{RC}}(3,2,1)$$
$$+\epsilon_{\mathtt{RC}}(3,2,1)+\epsilon_{\mathtt{RC}}(3\ell+3,3\ell+2,1)+\epsilon_{\mathtt{RC}}(2\ell+4,2\ell+3,1)=O\left(\frac{\ell k+N}{p-N}\right)$$

Additionally, in the batching $O(\ell)$ polynomials for the rowcheck, the soundness error occurs $O\left(\frac{\ell}{p-N}\right)$. Therefore, total soundness error is $O\left(\frac{\ell k+N}{p-N}\right)$.

– **Prover Time,** $\mathtt{Ptime}_{\mathtt{KG}}$:
The prover's actions consist of the followings

1. Sampling and Preprocessing Keys:
   Before the PIOP reduction, the prover samples and preprocessing keys using randomized encoding and NTT transform. The main overhead is $10\ell+8$ randomize encoding and its complexity is $O(\ell N\log N)$.
2. One norm check PIOP $\Pi_{\mathtt{NC}}$:
   Following the $\Pi_{\mathtt{NC}}$ prover time (Theorem 3), the prover time for a norm check PIOP is $\mathtt{Ptime}_{\mathtt{NC}}(5\ell+4,k)=O(\ell kN\log N)$, where $k=\lfloor\log B\rfloor+1$
3. Three linear check PIOPs $\Pi_{\mathtt{LC}}$:
   Following the $\Pi_{\mathtt{LC}}$ prover time (Theorem 2), the prover time for linear check PIOPs is as follows:

$$\begin{aligned}&\mathtt{Ptime}_{\mathtt{LC}}(5\ell+4,N^2)+\mathtt{Ptime}_{\mathtt{LC}}(1,N)+\mathtt{Ptime}_{\mathtt{LC}}(1,N)\\&\quad=O(\ell+N^2)+O(N\log N)+O(N\log N)\end{aligned}$$

   However, since the first linear check PIOP is designed to prove the linear relation involving the NTT matrix, the verifier cost can be reduced from $O(\ell+N^2)$ to $O(\ell+N\log N)$ by leveraging the NTT structure, even though the number of nonzero entries in the NTT matrix is $N^2$. Therefore, the verifier time complexity for linear check PIOPs is $O(\ell+N\log N)$.
4. Three row check PIOPs $\Pi_{\mathtt{RC}}$:
   To construct circuit of row check PIOPs, the prover computes $O(\ell N)$ operations. Following the $\Pi_{\mathtt{RC}}$ prover time (Theorem 1), the prover time for

linear check PIOPs is as follows:

$$\begin{aligned}&\mathtt{Ptime}_{\mathtt{RC}}(3,2,1)+\mathtt{Ptime}_{\mathtt{RC}}(3\ell+3,3\ell+2,1)+\mathtt{Ptime}_{\mathtt{RC}}(2\ell+4,2\ell+3,1)\\&\quad=O(N\log N)+O(\ell N\log N)+O(\ell N\log N)=O(\ell N\log N)\end{aligned}$$

Therefore, the total prover time is $O(\ell kN\log N)$.

– **Verifier Time,** $\mathtt{Vtime}_{\mathtt{KG}}$:
The verifier's actions consist of the followings.

1. One norm check PIOP $\Pi_{\mathtt{NC}}$:
   Following the $\Pi_{\mathtt{NC}}$ verifier time (Theorem 3), the verifier time for a norm check PIOP is $\mathtt{Vtime}_{\mathtt{NC}}(5\ell+4,k)=O(\ell kN\log N)$, where $k=\lfloor\log B\rfloor+1$
2. Three linear check PIOPs $\Pi_{\mathtt{LC}}$:
   Following the $\Pi_{\mathtt{LC}}$ verifier time (Theorem 2), the verifier time for linear check PIOPs is as follows:

$$\begin{aligned}&\mathtt{Vtime}_{\mathtt{LC}}(5\ell+4,N^2)+\mathtt{Vtime}_{\mathtt{LC}}(1,N)+\mathtt{Vtime}_{\mathtt{LC}}(1,N)\\&\quad=O(\ell+N^2)+O(N\log N)+O(N\log N)\end{aligned}$$

   However, since the first linear check PIOP is designed to prove the linear relation involving the NTT matrix, the verifier cost can be reduced from $O(\ell+N^2)$ to $O(\ell+N\log N)$ by leveraging the NTT structure, even though the number of nonzero entries in the NTT matrix is $N^2$. Therefore, the verifier time complexity for linear check PIOPs is $O(\ell+N\log N)$.
3. Three row check PIOPs $\Pi_{\mathtt{RC}}$:
   To construct circuit of row check PIOPs, the prover computes $O(\ell N)$ operations. Following the $\Pi_{\mathtt{RC}}$ prover time (Theorem 1), the prover time for linear check PIOPs is as follows:

$$\begin{aligned}&\mathtt{Vtime}_{\mathtt{RC}}(3,2,1)+\mathtt{Vtime}_{\mathtt{RC}}(3\ell+3,3\ell+2,1)+\mathtt{Vtime}_{\mathtt{RC}}(2\ell+4,2\ell+3,1)\\&\quad=O(N\log N)+O(\ell^2N\log N)+O(\ell^2\log N)=O(\ell^2\log N)\end{aligned}$$

The total verifier time is $O(\ell kN\log N+\ell^2\log N)$.

– **Query Complexity**:

1. One norm check PIOP $\Pi_{\mathtt{NC}}$:
   ity for Following the $\Pi_{\mathtt{NC}}$ query complexity (Theorem 3), the query complex a norm check PIOP is $(5\ell+4)k+5\ell+5$, where $k=\lfloor\log B\rfloor+1$
2. Three linear check PIOPs $\Pi_{\mathtt{LC}}$:
   Following the $\Pi_{\mathtt{LC}}$ query complexity (Theorem 2), the query complexity for linear check PIOPs is as follows:

$$\begin{aligned}&Q_{\mathtt{LC}}(5\ell+4,N^2)+Q_{\mathtt{LC}}(1,N)+Q_{\mathtt{LC}}(1,N)\\&\quad=(10\ell+11)+5+5=10\ell+21\end{aligned}$$

3. Three row check PIOPs $\Pi_{\mathtt{RC}}$:
   Following the $\Pi_{\mathtt{RC}}$ query complexity (Theorem 1), the query complexity for linear check PIOPs is as follows:

$$\begin{aligned}&Q_{\mathtt{RC}}(3,2,1)+Q_{\mathtt{RC}}(3\ell+3,3\ell+2,1)+Q_{\mathtt{RC}}(2\ell+4,2\ell+3,1)\\&\quad=3+(3\ell+3)+(2\ell+4)=5\ell+10\end{aligned}$$

The total query complexity is $(5\ell+4)k+20\ell+36$.

– **Size of Polynomial Oracles**:
  1. In step 1, prover sends $10\ell+8$ polynomial oracles, that are randomized encoding of keys. Each oracle has at most $L$ degree. Then, the size of polynomial oracles in step 1 is $(10\ell+8)L$ elements in $\mathbb{Z}_p$.
  2. One norm check PIOP $\Pi_{\mathtt{NC}}$: Following the $\Pi_{\mathtt{NC}}$ oracle sizes of (Theorem 3), the oracle size from a norm check PIOP is $\leq (5\ell k+3k+3)L$ elements in $\mathbb{Z}_p$, where $k=\lfloor \log B \rfloor+1$
  3. Three linear check PIOPs $\Pi_{\mathtt{LC}}$: $6L+6N$
  4. Three row check PIOPs $\Pi_{\mathtt{RC}}$ of degree 1: $3L$

  Therefore, the total size of polynomial oracles is $\leq (10\ell+5\ell k+3k+20)L+6N$ elements in $\mathbb{Z}_p$.
– **Size of witness**: The witness consists of $5\ell+3$ $R_p$ elements, which are identical to $\mathbb{Z}_p^N$. Thus, the witness size is $(5\ell+3)N$ elements in $\mathbb{Z}_p$.

### C.6 Analysis of $\Pi_{\mathtt{Enc}}$ (Proof of Theorem 5)

The PIOP $\Pi_{\mathtt{Enc}}$ is reduced to multiple PIOPs as follows: 2 norm check PIOP $\Pi_{\mathtt{NC}}$, 1 linear check PIOP $\Pi_{\mathtt{LC}}$, and 1 row check PIOP $\Pi_{\mathtt{RC}}$. Concretely, in step 3, $\Pi_{\mathtt{KG}}$ invokes two norm check PIOPs: one is for 3 witness vectors with $k_1$-length norm bound $B$, the other is for one witness vector with $k_2$-length norm bound $t$, i.e. $k_1=\lfloor \log B \rfloor+1$ and $k_2=\lfloor \log t \rfloor+1$. In step 4, it invokes linear check PIOP for 8 pairs of witnesses with NTT matrix $T$. In step 5, it invokes rowcheck PIOP for a degree-1 4 variable polynomial with 5 non-zero terms.

By the completeness, knowledge soundness, and HVZK properties of $\Pi_{\mathtt{LC}}$ (Theorem 2), $\Pi_{\mathtt{RC}}$ (Theorem 1), and $\Pi_{\mathtt{NC}}$ (Theorem 3), the PIOP $\Pi_{\mathtt{Enc}}$ in Fig. 5 achieves perfect completeness, knowledge soundness, and HVZK.
We consider the complexity analysis of the PIOP $\Pi_{\mathtt{Enc}}$.

– **Soundness Error,** $\epsilon_{\mathtt{Enc}}$:
  The sum of each soundness from PIOP invocations is as follows:

$$\begin{aligned}\epsilon_{\mathtt{NC}}(3,k_1)+\epsilon_{\mathtt{NC}}(1,k_2)+\epsilon_{\mathtt{LC}}(8,N^2)+\epsilon_{\mathtt{RC}}(5,4,1)\\ = O\left(\frac{k_1+k_2+N}{p-N}\right)\end{aligned}$$

  Additionally, in the batching 2 polynomials for the row check, the soundness error occurs $O\left(\frac{1}{p-N}\right)$. Therefore, the total soundness error is $O\left(\frac{k_1+k_2+N}{p-N}\right)$.
– **Prover Time,** $\mathtt{Ptime}_{\mathtt{Enc}}$:
  The prover's actions consist of the followings
  1. Sampling and Preprocessing:
     Before the PIOP reduction, the prover samples and preprocessing witness vectors using randomized encoding and NTT transform. The main overhead is 8 times randomize encoding and its complexity is $O(N\log N)$.

2. Two norm check PIOPs $\Pi_{\texttt{NC}}$:
   Following the $\Pi_{\texttt{NC}}$ prover time (Theorem 3), the prover time for a norm check PIOP is as follows:

$$\texttt{Ptime}_{\texttt{NC}}(3, k_1) + \texttt{Ptime}_{\texttt{NC}}(1, k_2) = O((k_1 + k_2)N \log N)$$

3. One linear check PIOP $\Pi_{\texttt{LC}}$:
   Following the $\Pi_{\texttt{LC}}$ prover time (Theorem 2), the prover time for linear check PIOPs is $\texttt{Ptime}_{\texttt{LC}}(8, N^2) = O(N^2)$. However, since the linear check PIOP is designed to prove the linear relation involving the NTT matrix, the prover cost can be reduced from $O(N^2)$ to $O(N \log N)$ by leveraging the NTT structure, even though the number of nonzero entries in the NTT matrix is $N^2$. Therefore, the verifier time complexity for linear check PIOPs is $O(N \log N)$.
4. One row check PIOP $\Pi_{\texttt{RC}}$:
   To construct the circuit for row check PIOP the prover performs $O(N \log N)$ operations. According to the prover time complexity for $\Pi_{\texttt{RC}}$ (Theorem 1), the prover time for row check PIOPs is $\texttt{Ptime}_{\texttt{RC}}(5, 4, 1) = O(N \log N)$.

Therefore, the total prover time is $O((k_1 + k_2)N \log N)$.

– **Verifier Time,** $\texttt{Vtime}_{\texttt{KG}}$:
The verifier's actions consist of the followings.

1. Two norm check PIOPs $\Pi_{\texttt{NC}}$:
   Following the $\Pi_{\texttt{NC}}$ verifier time (Theorem 3), the verifier time for a norm check PIOP is as follows:

$$\texttt{Vtime}_{\texttt{NC}}(3, k_1) + \texttt{Vtime}_{\texttt{NC}}(1, k_2) = O((k_1 + k_2)N \log N)$$

2. One linear check PIOP $\Pi_{\texttt{LC}}$:
   Following the $\Pi_{\texttt{LC}}$ prover time (Theorem 2), the verifier time for linear check PIOPs is $\texttt{Vtime}_{\texttt{LC}}(8, N^2) = O(N^2)$. However, since the linear check PIOP is designed to prove the linear relation involving the NTT matrix, the prover cost can be reduced from $O(N^2)$ to $O(N \log N)$ by leveraging the NTT structure, even though the number of nonzero entries in the NTT matrix is $N^2$. Therefore, the verifier time complexity for linear check PIOPs is $O(N \log N)$.
3. One row check PIOP $\Pi_{\texttt{RC}}$:
   To construct the circuit for row check PIOP the verifier performs $O(N \log N)$ operations. According to the verifier time complexity for $\Pi_{\texttt{RC}}$ (Theorem 1), the verifier time for row check PIOPs is $\texttt{Vtime}_{\texttt{RC}}(5, 4, 1) = O(N \log N)$.

The total verifier time is $O((k_1 + k_2)N \log N)$.

– **Query Complexity**:

1. Two norm check PIOPs $\Pi_{\texttt{NC}}$:
   Following the $\Pi_{\texttt{NC}}$ query complexity (Theorem 3), the query complex for norm check PIOPs is as follows:

$$Q_{\texttt{NC}}(3, k_1) + Q_{\texttt{NC}}(1, k_2) = 3k_1 + k_2 + 6$$

2. One linear check PIOP $\Pi_{\mathtt{LC}}$:
   Following the $\Pi_{\mathtt{LC}}$ query complexity (Theorem 2), the query complexity for a linear check PIOP is $Q_{\mathtt{LC}}(8, N^2) = 19$.
3. One row check PIOP $\Pi_{\mathtt{RC}}$:
   Following the $\Pi_{\mathtt{RC}}$ query complexity (Theorem 1), the query complexity for a row check PIOP is $Q_{\mathtt{RC}}(5, 4, 1) = 5$.

Therefore, the total query complexity is $3k_1 + k_2 + 30$.

– **Size of Polynomial Oracles**:
1. In step 1, prover sends 8 polynomial oracles, that are randomized encoding of witness vectors. Each oracle has at most $L$ degree. Then, the size of polynomial oracles in step 1 is $8L$.
2. Two norm check PIOPs $\Pi_{\mathtt{NC}}$: Following the $\Pi_{\mathtt{NC}}$ oracle sizes of (Theorem 3), the oracle size from norm check PIOPs is as follows:

$$\mathtt{Osize}_{\mathtt{NC}}(3, k_1) + \mathtt{Osize}_{\mathtt{NC}}(1, k_2) \leq (3k_1 + k_2 + 6)L$$

3. One linear check PIOP $\Pi_{\mathtt{LC}}$: Following the $\Pi_{\mathtt{LC}}$ oracle sizes of (Theorem 2), the oracle size from a linear check PIOP is $\mathtt{Osize}_{\mathtt{LC}}(8, N^2) = 2L + 2N$
4. One row check PIOP $\Pi_{\mathtt{RC}}$: Following the $\Pi_{\mathtt{RC}}$ oracle sizes of (Theorem 1), the oracle size from a row check PIOP is $\mathtt{Osize}_{\mathtt{RC}}(5, 4, 1) = L$

Therefore, the total size of polynomial oracles is $\leq (3k_1 + k_2 + 17)L + 2N$ elements in $\mathbb{Z}_p$

– **Size of witness**: The witness consists of 4 $R_p$ elements, which are identical to $\mathbb{Z}_p^N$. Thus, the witness size is $4N$ elements in $\mathbb{Z}_p$.

### C.7 Analysis of $\Pi_{\mathtt{DD}}$ (Proof of Theorem 6)

The PIOP $\Pi_{\mathtt{DD}}$ is reduced to multiple PIOPs as follows: 2 norm check PIOP $\Pi_{\mathtt{NC}}$, 1 linear check PIOP $\Pi_{\mathtt{LC}}$, and 1 row check PIOP $\Pi_{\mathtt{RC}}$. Concretely, in step 3, $\Pi_{\mathtt{KG}}$ invokes two norm check PIOPs: one is for 3 witness vectors with $k_1$-length norm bound $B$, the other is for one witness vector with $k_2$-length norm bound $t$, i.e. $k_1 = \lfloor \log B \rfloor + 1$ and $k_3 = \lfloor \log B(N+1) \rfloor + 1$. In step 4, it invokes linear check PIOP for 8 pairs of witnesses with NTT matrix $T$. In step 5, it invokes rowcheck PIOP for a degree-1 4 variable polynomial with 5 non-zero terms.

By the completeness, knowledge soundness, and HVZK properties of $\Pi_{\mathtt{LC}}$ (Theorem 2), $\Pi_{\mathtt{RC}}$ (Theorem 1), and $\Pi_{\mathtt{NC}}$ (Theorem 3), the PIOP $\Pi_{\mathtt{DD}}$ in Fig. 6 achieves perfect completeness, knowledge soundness, and HVZK.
We consider the complexity analysis of the PIOP $\Pi_{\mathtt{DD}}$.

– **Soundness Error, $\epsilon_{\mathtt{DD}}$**:
The sum of each soundness from PIOP invocations is as follows:

$$\epsilon_{\mathtt{NC}}(4, k_1) + \epsilon_{\mathtt{NC}}(1, k_3) + \epsilon_{\mathtt{LC}}(10, N^2) + \epsilon_{\mathtt{RC}}(6, 5, 1) = O(\frac{k_1 + k_3 + N}{p - N})$$

Additionally, in the batching 2 polynomials for the rowcheck, the soundness error occurs $O(\frac{1}{p-N})$. Therefore, total soundness error is $O(\frac{k_1+k_3+N}{p-N})$.

– **Prover Time,** $\texttt{Ptime}_{\texttt{DD}}$:
The prover's actions consist of the followings
1. Sampling and Preprocessing:
Before the PIOP reduction, the prover samples and preprocessing witness vectors using randomized encoding and NTT transform. The main overhead is 10 times randomize encoding and its complexity is $O(N \log N)$.
2. Two norm check PIOPs $\Pi_{\texttt{NC}}$:
Following the $\Pi_{\texttt{NC}}$ prover time (Theorem 3), the prover time for a norm check PIOP is as follows:

$$\texttt{Ptime}_{\texttt{NC}}(4, k_1) + \texttt{Ptime}_{\texttt{NC}}(1, k_3) = O((k_1 + k_3)N \log N)$$

3. One linear check PIOP $\Pi_{\texttt{LC}}$:
Following the $\Pi_{\texttt{LC}}$ prover time (Theorem 2), the prover time for linear check PIOPs is $\texttt{Ptime}_{\texttt{LC}}(10, N^2) = O(N^2)$. However, since the linear check PIOP is designed to prove the linear relation involving the NTT matrix, the prover cost can be reduced from $O(N^2)$ to $O(N \log N)$ by leveraging the NTT structure, even though the number of nonzero entries in the NTT matrix is $N^2$. Therefore, the verifier time complexity for linear check PIOPs is $O(N \log N)$.
4. One row check PIOP $\Pi_{\texttt{LC}}$:
To construct the circuit for row check PIOP the prover performs $O(N \log N)$ operations. According to the prover time complexity for $\Pi_{\texttt{RC}}$ (Theorem 1), the prover time for row check PIOPs is $\texttt{Ptime}_{\texttt{RC}}(6, 5, 1) = O(N \log N)$.

Therefore, the total prover time is $O((k_1 + k_3)N \log N)$.

– **Verifier Time,** $\texttt{Vtime}_{\texttt{KG}}$:
The verifier's actions consist of the following.
1. Two norm check PIOPs $\Pi_{\texttt{NC}}$:
Following the $\Pi_{\texttt{NC}}$ verifier time (Theorem 3), the verifier time for a norm check PIOP is as follows:

$$\texttt{Vtime}_{\texttt{NC}}(4, k_1) + \texttt{Vtime}_{\texttt{NC}}(1, k_3) = O((k_1 + k_3)N \log N)$$

2. One linear check PIOP $\Pi_{\texttt{LC}}$:
Following the $\Pi_{\texttt{LC}}$ prover time (Theorem 2), the verifier time for linear check PIOPs is $\texttt{Vtime}_{\texttt{LC}}(10, N^2) = O(N^2)$. However, since the linear check PIOP is designed to prove the linear relation involving the NTT matrix, the prover cost can be reduced from $O(N^2)$ to $O(N \log N)$ by leveraging the NTT structure, even though the number of nonzero entries in the NTT matrix is $N^2$. Therefore, the verifier time complexity for linear check PIOPs is $O(N \log N)$.
3. One row check PIOP $\Pi_{\texttt{RC}}$:
To construct the circuit for row check PIOP the verifier performs $O(N \log N)$ operations. According to the verifier time complexity for $\Pi_{\texttt{RC}}$ (Theorem 1), the verifier time for row check PIOPs is $\texttt{Vtime}_{\texttt{RC}}(6, 5, 1) = O(N \log N)$.

Therefore, the total verifier time is $O((k_1 + k_2)N \log N)$.

– **Query Complexity**:

1. Two times normcheck PIOP $\Pi_{\mathtt{NC}}$:
   Following the $\Pi_{\mathtt{NC}}$ query complexity (Theorem 3), the query complex for norm check PIOPs is as follows:

$$Q_{\mathtt{NC}}(4, k_1) + Q_{\mathtt{NC}}(1, k_3) = (4k_1 + 5) + (k_3 + 2) = 3k_1 + k_2 + 6$$

2. One linear check PIOP $\Pi_{\mathtt{LC}}$:
   Following the $\Pi_{\mathtt{LC}}$ query complexity (Theorem 2), the query complexity for a linear check PIOP is $Q_{\mathtt{LC}}(10, N^2) = 23$.
3. One row check PIOP $\Pi_{\mathtt{RC}}$:
   Following the $\Pi_{\mathtt{RC}}$ query complexity (Theorem 1), the query complexity for a row check PIOP is $Q_{\mathtt{RC}}(6, 5, 1) = 6$.

Therefore, the total query complexity is $4k_1 + k_3 + 36$.

– **Size of Polynomial Oracles**:
1. In step 1, prover sends 10 polynomial oracles, that are randomized encoding of witness vectors. Each oracle has at most $L$ degree. Then, the size of polynomial oracles in step 1 is $10L$.
2. Two times normcheck PIOP $\Pi_{\mathtt{NC}}$: Following the $\Pi_{\mathtt{NC}}$ oracle sizes of (Theorem 3), the oracle size from norm check PIOPs is as follows:

$$\mathtt{Osize}_{\mathtt{NC}}(4, k_1) + \mathtt{Osize}_{\mathtt{NC}}(1, k_3) \leq (4k_1 + k_3 + 6)L$$

3. One linear check PIOP $\Pi_{\mathtt{LC}}$: Following the $\Pi_{\mathtt{LC}}$ oracle sizes of (Theorem 2), the oracle size from a linear check PIOP is $\mathtt{Osize}_{\mathtt{LC}}(10, N^2) \leq 2L + 2N$
4. One row check PIOP $\Pi_{\mathtt{RC}}$: Following the $\Pi_{\mathtt{RC}}$ oracle sizes of (Theorem 1), the oracle size from a row check PIOP is $\mathtt{Osize}_{\mathtt{RC}}(6, 5, 1) \leq L$

Therefore, the total size of polynomial oracles is $\leq (4k_1 + k_3 + 19)L + 2N$ elements in $\mathbb{Z}_p$

– **Size of witness**: The witness consists of 4 $R_p$ elements, which are identical to $\mathbb{Z}_p^N$. Thus, the size of witness is $4N$ elements in $\mathbb{Z}_p$.

### C.8 Analysis of Modulus Switching (Proof of Lemmas 1 to 3)

Throughout the proof, we write the rounding error of $x$ as $\varepsilon_x := \left\lfloor \frac{q}{p}x \right\rceil - \frac{q}{p}x$. Note that $\|x\|_\infty \leq \frac{1}{2}$.

Moreover, we use the fact that $\left\|\vec{g} - \frac{q}{p}\vec{g}'\right\|_\infty \leq \frac{q}{2p}$, since $\frac{q}{q_i} - \frac{q}{p}\left\lfloor \frac{p}{q_i} \right\rceil = \frac{q}{q_i} - \frac{q}{p}\left(\frac{p}{q_i} + \varepsilon\right) = -\frac{q}{p}\varepsilon$ where $|\varepsilon| \leq \frac{1}{2}$.

**Proof of Lemma 1.** We have

$$\begin{aligned}
\left\|\vec{\boldsymbol{e}}'_{\mathsf{rlk},0}\right\|_\infty &= \left\| \left\lfloor \frac{q}{p}\vec{\boldsymbol{r}}_0 \right\rceil + \boldsymbol{s} \left\lfloor \frac{q}{p}\vec{\boldsymbol{u}}_{\mathsf{rlk},0} \right\rceil \right\|_\infty \\
&= \left\| \frac{q}{p}(\vec{\boldsymbol{r}}_0 + \boldsymbol{s}\vec{\boldsymbol{u}}_{\mathsf{rlk},0}) + \varepsilon_{\vec{\boldsymbol{r}}_0} + \boldsymbol{s}\varepsilon_{\vec{\boldsymbol{u}}_{\mathsf{rlk},0}} \right\|_\infty \\
&= \left\| \frac{q}{p}\vec{\boldsymbol{e}}_{\mathsf{rlk},0} + \varepsilon_{\vec{\boldsymbol{r}}_0} + \boldsymbol{s}\varepsilon_{\vec{\boldsymbol{u}}_{\mathsf{rlk},0}} \right\|_\infty \\
&\leq \frac{q}{p}\|\vec{\boldsymbol{e}}_{\mathsf{rlk},0}\|_\infty + \frac{1}{2}(NB + 1)
\end{aligned}$$

$$
\begin{aligned}
\left\|\vec{\boldsymbol{e}}'_{\mathsf{rlk},1}\right\|_\infty &= \left\| \left\lfloor \frac{q}{p}\vec{\boldsymbol{r}}_1 \right\rceil + \boldsymbol{f}_{\mathsf{rlk}} \left\lfloor \frac{q}{p}\vec{\boldsymbol{u}}_{\mathsf{rlk},0} \right\rceil - \boldsymbol{s}\cdot\vec{g} \right\|_\infty \\
&= \left\| \frac{q}{p}(\vec{\boldsymbol{r}}_1 + \boldsymbol{f}_{\mathsf{rlk}}\vec{\boldsymbol{u}}_{\mathsf{rlk},0}) + \varepsilon_{\vec{\boldsymbol{r}}_1} + \boldsymbol{f}_{\mathsf{rlk}}\varepsilon_{\vec{\boldsymbol{u}}_{\mathsf{rlk},0}} - \boldsymbol{s}\cdot\vec{g} \right\|_\infty \\
&= \left\| \frac{q}{p}(\boldsymbol{s}\cdot\vec{g}' + \vec{\boldsymbol{e}}_{\mathsf{rlk},1}) + \varepsilon_{\vec{\boldsymbol{r}}_1} + \boldsymbol{f}_{\mathsf{rlk}}\varepsilon_{\vec{\boldsymbol{u}}_{\mathsf{rlk},0}} - \boldsymbol{s}\cdot\vec{g} \right\|_\infty \\
&= \left\| \frac{q}{p}\vec{\boldsymbol{e}}_{\mathsf{rlk},1} + \varepsilon_{\vec{\boldsymbol{r}}_1} + \boldsymbol{f}_{\mathsf{rlk}}\varepsilon_{\vec{\boldsymbol{u}}_{\mathsf{rlk},0}} - \boldsymbol{s}\cdot(\vec{g} - \frac{q}{p}\vec{g}') \right\|_\infty \\
&\le \frac{q}{p}\|\vec{\boldsymbol{e}}_{\mathsf{rlk},1}\|_\infty + \frac{1}{2}\left(\frac{q}{p}B + NB + 1\right)
\end{aligned}
$$

$$
\begin{aligned}
\|\vec{\boldsymbol{e}}_{\mathsf{rlk},2}\|_\infty &= \left\| \left\lfloor \frac{q}{p}\vec{\boldsymbol{r}}_2 \right\rceil + \boldsymbol{s} \left\lfloor \frac{q}{p}\vec{\boldsymbol{u}}_{\mathsf{rlk},1} \right\rceil + \boldsymbol{f}_{\mathsf{rlk}}\cdot\vec{g} \right\|_\infty \\
&= \left\| \frac{q}{p}(\vec{\boldsymbol{r}}_2 + \boldsymbol{s}\vec{\boldsymbol{u}}_{\mathsf{rlk},1}) + \varepsilon_{\vec{\boldsymbol{r}}_2} + \boldsymbol{s}\varepsilon_{\vec{\boldsymbol{u}}_{\mathsf{rlk},1}} + \boldsymbol{f}_{\mathsf{rlk}}\cdot\vec{g} \right\|_\infty \\
&= \left\| \frac{q}{p}(-\boldsymbol{f}_{\mathsf{rlk}}\cdot\vec{g}' + \vec{\boldsymbol{e}}_{\mathsf{rlk},2}) + \varepsilon_{\vec{\boldsymbol{r}}_2} + \boldsymbol{f}_{\mathsf{rlk}}\varepsilon_{\vec{\boldsymbol{u}}_{\mathsf{rlk},1}} + \boldsymbol{f}_{\mathsf{rlk}}\cdot\vec{g} \right\|_\infty \\
&= \left\| \frac{q}{p}\vec{\boldsymbol{e}}_{\mathsf{rlk},2} + \varepsilon_{\vec{\boldsymbol{r}}_2} + \boldsymbol{s}\varepsilon_{\vec{\boldsymbol{u}}_{\mathsf{rlk},1}} + \boldsymbol{f}_{\mathsf{rlk}}\cdot(\vec{g} - \frac{q}{p}\vec{g}') \right\|_\infty \\
&\le \frac{q}{p}\|\vec{\boldsymbol{e}}_{\mathsf{rlk},2}\|_\infty + \frac{1}{2}\left(\frac{q}{p}B + NB + 1\right)
\end{aligned}
$$

Therefore, $\|\vec{\boldsymbol{e}}_{\mathsf{rlk}}\|_\infty \le \frac{q}{p}\|\vec{\boldsymbol{e}}_{\mathsf{rlk}}\|_\infty + \frac{1}{2}\left(\frac{q}{p}B + NB + 1\right)$.

**Proof of Lemma 2.** We have

$$
\begin{aligned}
\left\|\vec{\boldsymbol{e}}'_{\mathsf{atk},0}\right\|_\infty &= \left\| \left\lfloor \frac{q}{p}\vec{\boldsymbol{a}}_0 \right\rceil + \boldsymbol{s} \left\lfloor \frac{q}{p}\vec{\boldsymbol{u}}_{\mathsf{atk},0} \right\rceil - \boldsymbol{s}(X^{-1})\cdot\vec{g} \right\|_\infty \\
&= \left\| \frac{q}{p}(\vec{\boldsymbol{a}}_0 + \boldsymbol{s}\vec{\boldsymbol{u}}_{\mathsf{atk},0}) + \varepsilon_{\vec{\boldsymbol{a}}_0} + \boldsymbol{s}\varepsilon_{\vec{\boldsymbol{u}}_{\mathsf{atk},0}} - \boldsymbol{s}(X^{-1})\cdot\vec{g} \right\|_\infty \\
&= \left\| \frac{q}{p}\vec{\boldsymbol{e}}_{\mathsf{atk},0} + \varepsilon_{\vec{\boldsymbol{a}}_0} + \boldsymbol{s}\varepsilon_{\vec{\boldsymbol{u}}_{\mathsf{atk},0}} - \boldsymbol{s}(X^{-1})\cdot(\vec{g} - \frac{q}{p}\vec{g}') \right\|_\infty \\
&\le \frac{q}{p}\|\vec{\boldsymbol{e}}_{\mathsf{atk},0}\|_\infty + \frac{1}{2}\left(\frac{q}{p}B + NB + 1\right).
\end{aligned}
$$

Similarly, $\left\|\vec{\boldsymbol{e}}'_{\mathsf{atk},1}\right\|_\infty \le \frac{q}{p}\|\vec{\boldsymbol{e}}_{\mathsf{atk},0}\|_\infty + \frac{1}{2}\left(\frac{q}{p}B + NB + 1\right)$. Therefore, $\left\|\vec{\boldsymbol{e}}'_{\mathsf{atk}}\right\|_\infty \le \frac{q}{p}\|\vec{\boldsymbol{e}}_{\mathsf{atk}}\|_\infty + \frac{1}{2}\left(\frac{q}{p}B + NB + 1\right)$.

**Proof of Lemma 3.** If $\mathsf{ek} = \boldsymbol{p} \in R_p$ was verified through $\Pi_{\mathtt{DKG}}$, then we have $\boldsymbol{p} = -\boldsymbol{u}_{\mathsf{ek}}\boldsymbol{s} + \boldsymbol{e}_{\mathsf{ek}} \pmod p$, where $\|\boldsymbol{s}\|_\infty := \left\|\sum_{i\in I}\boldsymbol{s}^{(i)}\right\|_\infty \le B|I|$ and $\|\boldsymbol{e}_{\mathsf{ek}}\|_\infty \le B|I|$. Therefore, $\mathsf{ct} = (\boldsymbol{c}_0, \boldsymbol{c}_1) \in R_p^2$ satisfies

$$
\begin{aligned}
(\boldsymbol{c}_0, \boldsymbol{c}_1) &= \boldsymbol{f}\cdot(-\boldsymbol{u}_{\mathsf{ek}}\boldsymbol{s} + \boldsymbol{e}_{\mathsf{ek}}, \boldsymbol{u}_{\mathsf{ek}}) + (\lfloor p/t\rceil\cdot\boldsymbol{m} + \boldsymbol{e}_0, \boldsymbol{e}_1) \\
&= (-\boldsymbol{f}\boldsymbol{u}_{\mathsf{ek}}\boldsymbol{s} + \boldsymbol{f}\boldsymbol{e}_{\mathsf{ek}} + \lfloor p/t\rceil\cdot\boldsymbol{m} + \boldsymbol{e}_0, \boldsymbol{f}\boldsymbol{u}_{\mathsf{ek}} + \boldsymbol{e}_1) \pmod p
\end{aligned}
$$

hence $\boldsymbol{c}_0 + \boldsymbol{c}_1\boldsymbol{s} = \lfloor p/t \rceil \cdot \boldsymbol{m} + \boldsymbol{e}_1\boldsymbol{s} + \boldsymbol{f}\boldsymbol{e}_{\mathsf{ek}} + \boldsymbol{e}_0 \pmod p$. This implies that

$$\begin{aligned}
\|\boldsymbol{e}'\|_\infty &= \left\| \left\lfloor \frac{q}{p}\boldsymbol{c}_0 \right\rceil + \left\lfloor \frac{q}{p}\boldsymbol{c}_1 \right\rceil \boldsymbol{s} - \left\lfloor \frac{q}{t} \right\rceil \cdot \boldsymbol{m} \right\|_\infty \\
&= \left\| \frac{q}{p}(\boldsymbol{c}_0 + \boldsymbol{c}_1\boldsymbol{s}) + \varepsilon_{\boldsymbol{c}_0} + \varepsilon_{\boldsymbol{c}_1}\boldsymbol{s} - \left\lfloor \frac{q}{t} \right\rceil \cdot \boldsymbol{m} \right\|_\infty \\
&= \left\| \frac{q}{p}(\boldsymbol{e}_1\boldsymbol{s} + \boldsymbol{f}\boldsymbol{e}_{\mathsf{ek}} + \boldsymbol{e}_0) + \varepsilon_{\boldsymbol{c}_0} + \varepsilon_{\boldsymbol{c}_1}\boldsymbol{s} + \left( \frac{q}{p} \left\lfloor \frac{p}{t} \right\rceil - \left\lfloor \frac{q}{t} \right\rceil \right) \cdot \boldsymbol{m} \right\|_\infty \\
&\leq \frac{q}{p}(2NB^2 + B)|I| + \frac{1}{2}\left( \frac{q}{p} 2t + NB|I| + 1 \right).
\end{aligned}$$

# D Parameters for Polynomial Commitment

In the HSS scheme, we configure the parameters $\log \mathsf{q}$, $\mathsf{N}$, $\mathsf{n}$, $\mathsf{d}$, $\mu$, and $\nu$ for the evaluation protocol. For detailed definitions, see Table 1 in [HSS24]. The input size $\mathsf{N}$ is calculated from Theorems 4 to 6, and all polynomials are evaluated in a batched manner.

| | $\lceil \log \mathsf{q} \rceil$ | $\mathsf{N}$ | $\mathsf{n}$ | $\mathsf{d}$ | $\mu$ | $\nu$ |
|---|---|---|---|---|---|---|
| Params I (Setup) | 110 | $\approx 221 \cdot 2^{14}$ | $2^{13}$ | $2^{12}$ | 1 | 1 |
| Params I (Enc) | 110 | $\approx 47 \cdot 2^{14}$ | $2^{12}$ | $2^{12}$ | 1 | 1 |
| Params I (Dec) | 110 | $\approx 50 \cdot 2^{14}$ | $2^{12}$ | $2^{12}$ | 1 | 1 |
| Params I (PELTA) | 110 | $\approx 17 \cdot 2^{14}$ | $2^{12}$ | $2^{12}$ | 1 | 1 |
| Params II (Setup) | 110 | $\approx 401 \cdot 2^{15}$ | $2^{14}$ | $2^{12}$ | 1 | 1 |
| Params II (Enc) | 110 | $\approx 47 \cdot 2^{15}$ | $2^{12}$ | $2^{12}$ | 1 | 1 |
| Params II (Dec) | 110 | $\approx 51 \cdot 2^{15}$ | $2^{12}$ | $2^{12}$ | 1 | 1 |
| Params II (PELTA) | 110 | $\approx 17 \cdot 2^{15}$ | $2^{12}$ | $2^{12}$ | 1 | 1 |

Table 6: Parameters for the HSS scheme